第八章

查　找

何谓查找表？

查找表是由**同一类型**的数据元素(或记录)构成的**集合**。

由于“集合”中的数据元素之间存在着松散的关系，因此查找表是一种应用灵便的结构。

对查找表经常进行的操作:

- 1）**查询**某个“特定的”数据元素是否在查找表中；
- 2）**检索**某个“特定的”数据元素的各种属性；
- 3）在查找表中**插入**一个数据元素；
- 4）从查找表中**删去**某个数据元素。

查找表可分为两类：

- **静态查找表**

  仅作查询和检索操作的查找表。

- **动态查找表**

有时在查询之后，还需要将“查询”结果为“不在查找表中”的数据元素插入到查找表中；或者，从查找表中删除其“查询”结果为“在查找表中”的数据元素。

关键字

是数据元素（或记录）中某个**数据项**的值，用以**标识**（识别）一个数据元素（或记录）。

若此关键字可以识别**唯一的**一个记录，则称之谓**“主关键字”**。

若此关键字能识别**若干**记录，则称之谓**“次关键字”**。

查找

根据给定的某个值，在查找表中确定一个其关键字等于给定值的数据元素或（记录）。

若查找表中存在这样一个记录，则称“**查找成功**”。查找结果给出整个记录的信息，或指示该记录在查找表中的位置；

否则称“**查找不成功**”。查找结果给出

“空记录”或“空指针”

如何进行查找?

查找的方法取决于查找表的结构。

由于查找表中的数据元素之间不存在明显的组织规律，因此不便于查找。

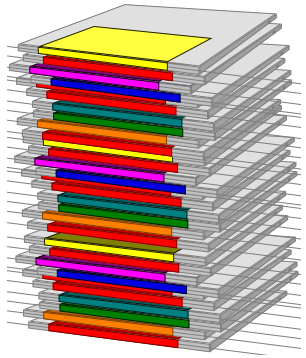
为了提高查找的效率， 需要在查找表中的元素之间人为地 附加某种确定的关系，换句话说,**用另外一种结构来表示查找表**。

9.1 静态查找表

9.2 动态查找树表

9.3 哈希表





9.1

静态查找表

ADT StaticSearchTable {

数据对象D：D是具有相同特性的数据元素的集合。**每个数据元素含有类型相同的关键字**，可唯一标识数据元素。

数据关系R：数据元素同属一个集合。

基本操作 P：

Create(&ST, n);

Destroy(&ST);

Search(ST, key);

Traverse(ST, Visit());

} ADT StaticSearchTable



Create(&ST, n);

操作结果：构造一个含n个数据元素的静态查找表ST。



Destroy(&ST);

初始条件：静态查找表ST存在；

操作结果：销毁表ST。



Search(ST, key);

初始条件：静态查找表ST存在，key为和查找表中元素的关键字类型相同的给定值；

操作结果：若 ST 中存在其关键字等于 key 的数据元素，则函数值为该元素的值或在表中的位置，否则为“空”。



Traverse(ST, Visit());

初始条件：静态查找表ST存在，Visit是对元素操作的应用函数；

操作结果：按某种次序对ST的每个元素调用函数Visit()一次且仅一次，一旦Visit()失败，则操作失败。



假设**静态查找表**的**顺序存储结构**为

```
typedef  struct {
   ElemType *elem;
        // 数据元素存储空间基址，建表时
        // 按实际长度分配，0号单元留空
   int       length;     // 表的长度
} SSTable;
```



数据元素类型的定义为:

```
typedef struct {
    keyType key;     // 关键字域
    ... ...                  // 其它属性域
} ElemTyp  , TElemType ;
```



一、顺序查找表

二、有序查找表

三、静态查找树表

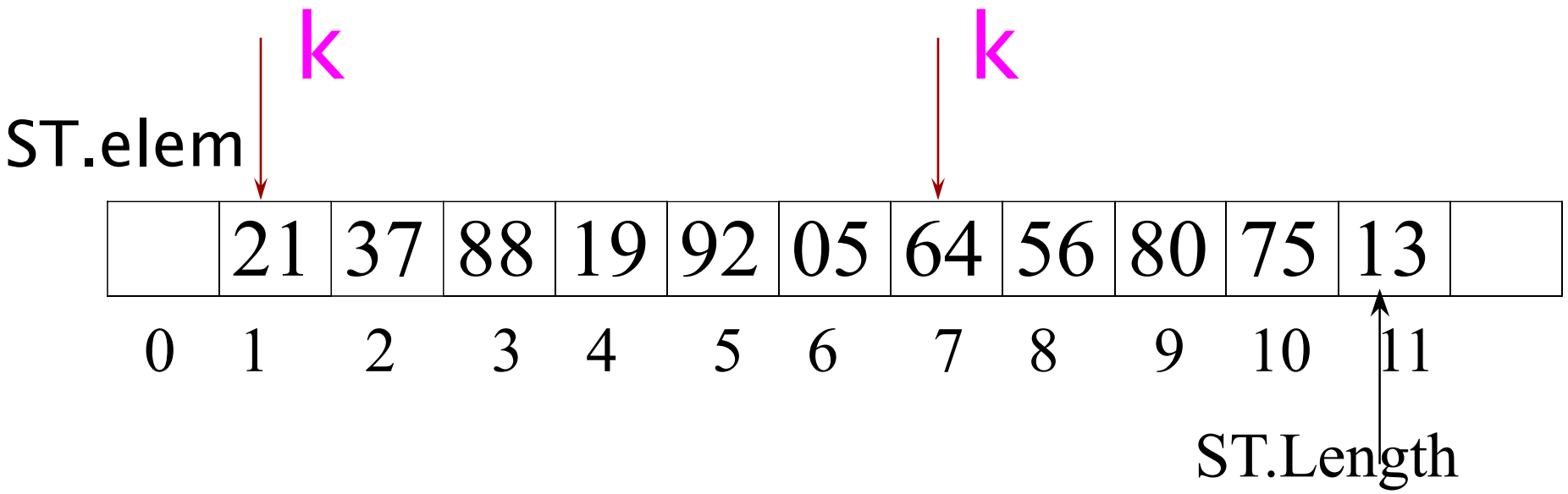
四、索引顺序表



一、顺序查找表

以顺序表或线性链表表示静态查找表

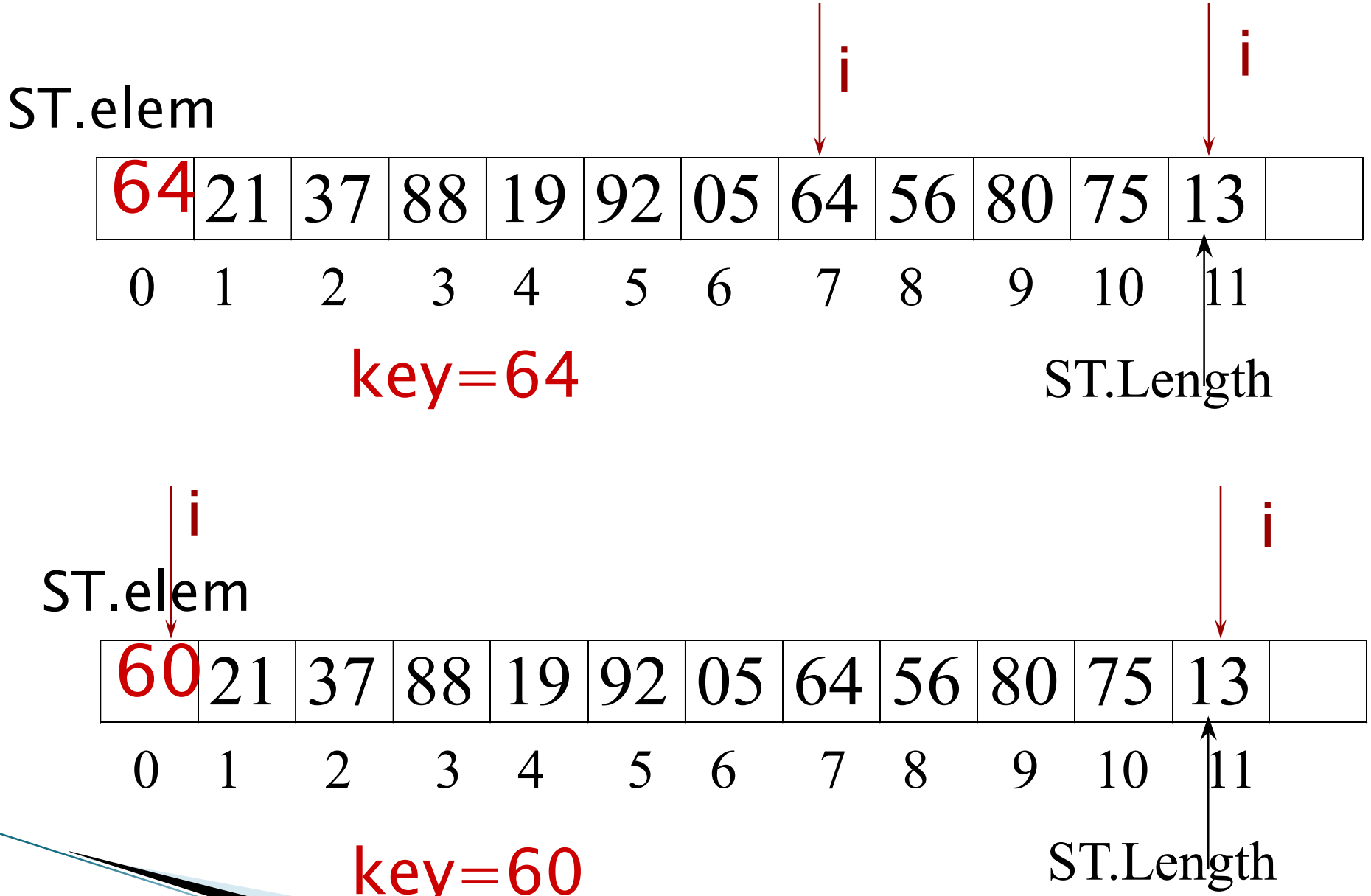
回顾顺序表的查找过程：



假设给定值 e=64,

要求 ST.elem[k] = e, 问: k = ?

```
int location( SqList L, ElemType& e,
        Status (*compare)(ElemType, ElemType))
  k = 1;
  p = L.elem;
  while ( k<=L.length &&
                !(*compare)(*p++,e)))
  if ( k<= L.length)  return k;
  else  return 0;
} //location
```


ST.elem
i
i
64 21 37 88 19 92 05 64 56 80 75 13
0 1 2 3 4 5 6 7 8 9 10 11
key=64
ST.Length
i
i
ST.elem
60 21 37 88 19 92 05 64 56 80 75 13
0 1 2 3 4 5 6 7 8 9 10 11
key=60
ST.Length

```
int Search_Seq(SSTable ST,
                                KeyType ke
    // 在顺序表ST中顺序查找其关键字等于
    //  key的数据元素。若找到，则函数值为
    // 该元素在表中的位置，否则为0。
    ST.elem[0].key = key;          // “哨兵”
    for (i=ST.length; ST.elem[i].key!=key;  --
                                   // 从后往前找
    return i;                  // 找不到时，i为0
} // Search_Seq
```

分析顺序查找的时间性能

定义： 查找算法的**平均查找长度**

(**A**verage **S**earch **L**ength)

为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值

$$ASL = \sum_{i=1}^{n} P_i C_i$$

其中：n 为表长，P_i 为查找表中第i个记录的概率，且 $\sum_{i=1}^{n} P_i = 1$ ，C_i为找到该记录时，曾和给定值比较过的关键字的个数。

对**顺序表**而言，$C_i = n-i+1$

$ASL = nP_1 + (n-1)P_2 + \ + 2P_{n-1} +$

在等概率查找的情况下，$P_i = \frac{1}{n}$

顺序表查找的平均查找长度为：

$$ASL_{ss} = \frac{1}{n}\sum_{i=1}^{n}(n-i+1) = \frac{n+1}{2}$$

在不等概率查找的情况下，ASL_{ss} 在

$$P_n \geq P_{n-1} \geq \cdots \geq P_2 \geq P_1$$

时取极小值

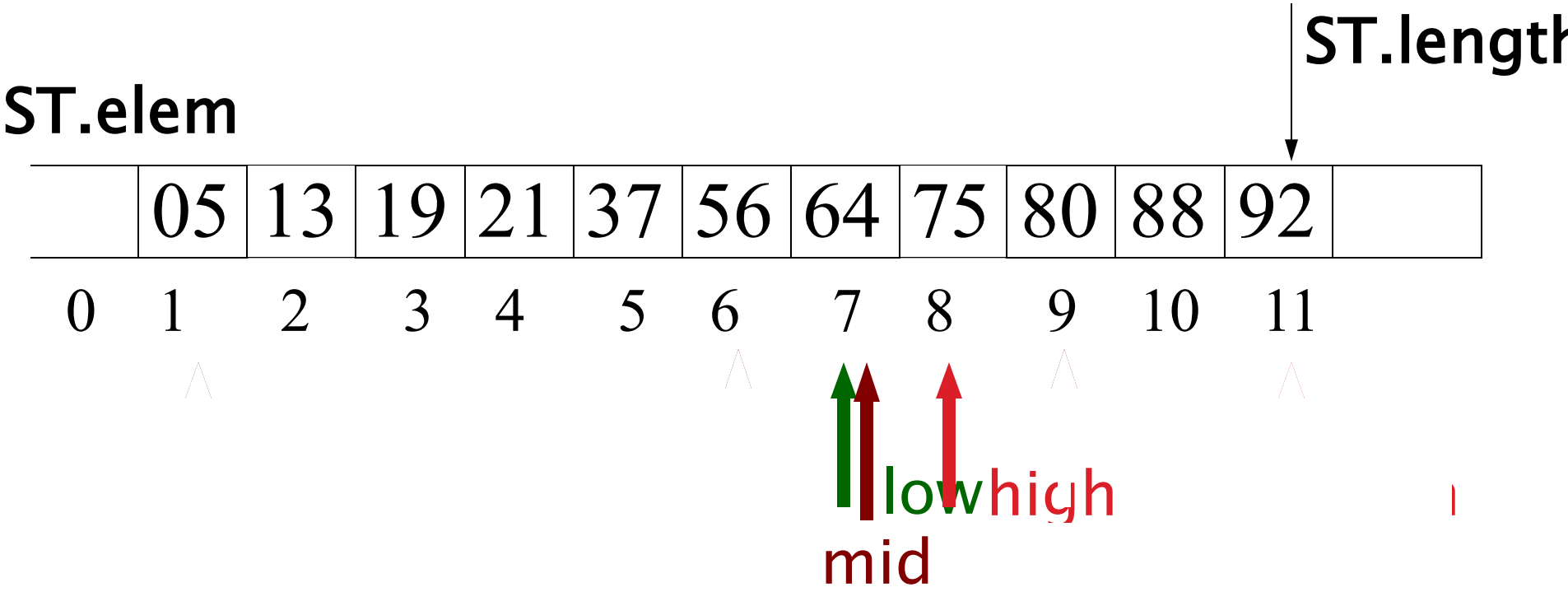
若查找概率无法事先测定，则查找过程采取的改进办法是，在每次查找之后，将刚刚查找到的记录直接移至表尾的位置上。

二、有序查找表

上述顺序查找表的查找算法简单，但平均查找长度较大，特别不适用于表长较大的查找表。

若以**有序表**表示静态查找表，则查找过程可以基于“**折半**”进行。

例如：key=64 的查找过程如下：



low 指示查找区间的下界

high 指示查找区间的上界

mid = (low+high)/2

```
int Search_Bin ( SSTable ST, KeyType key
   low = 1;  high = ST.length;      // 置区间初
   while (low <= high) { // 待查区间含 low 和 high
      mid = (low + high) / 2;
      if   (EQ (key , ST.elem[mid].key) )
        return  mid;            // 找到待查元素
      else  if ( LT (key , ST.elem[mid].key)
        high = mid - 1;          // 继续在前半区间进
      else  low = mid + 1; // 继续在后半区间进
   }
   return 0;                    // 顺序表中不存在待查元
} // Search_Bin
```

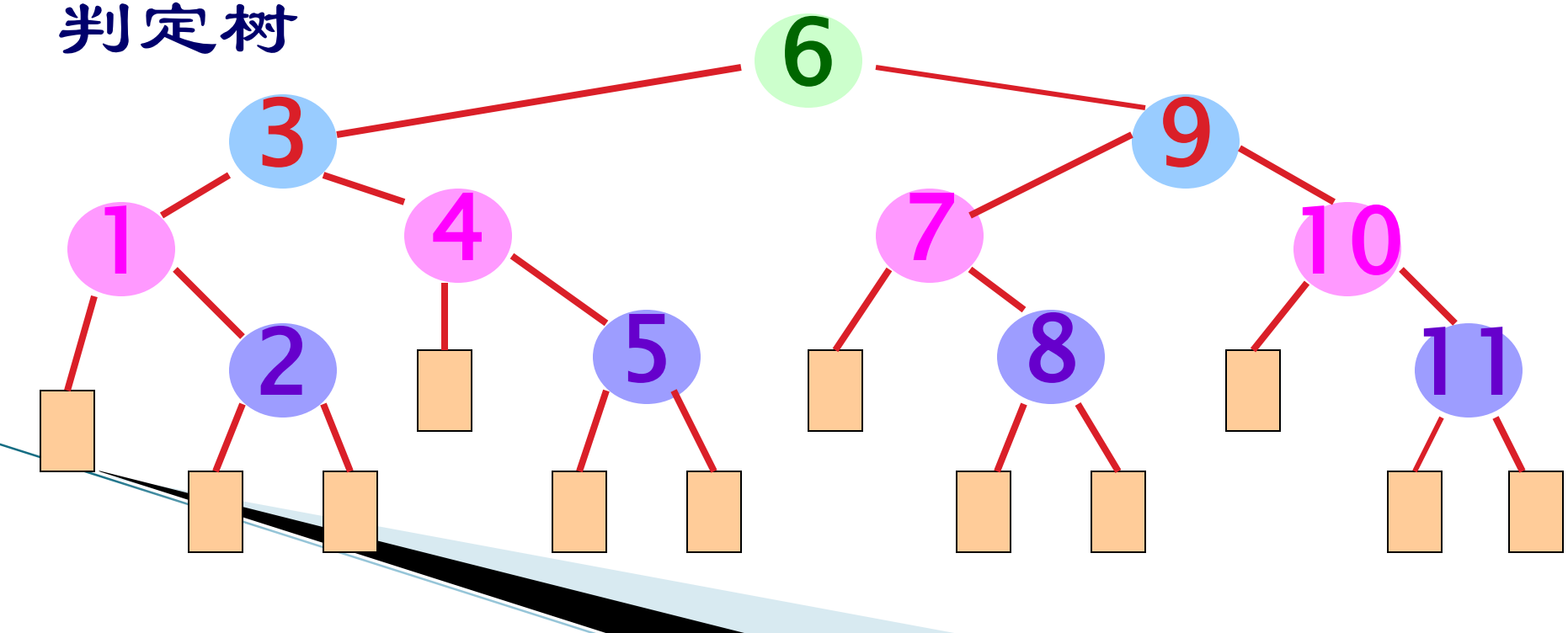
分析**折半查找**的平均查找长度

先看一个具体的情况，假设：

n=11

i	1	2	3	4	5	6	7	8	9	10	11
Ci	3	4	2	3	4	1	3	4	2	3	4

判定树



一般情况下，表长为 n 的折半查找的判定树的深度和含有 n 个结点的完全二叉树的深度相同。

假设 $n=2^h-1$ 并且查找概率相等

则 $$ASL_{bs}=\frac{1}{n}\sum_{i=1}^{n}C_i=\frac{1}{n}\left[\sum_{j=1}^{h}j\times 2^{j-1}\right]=\frac{n+1}{n}\log_2(n+1)-1$$

在n>50时，可得近似结果

$$ASL_{bs}\approx\log_2(n+1)-1$$

三、静态查找树表

在**不等概率查找**的情况下，**折半查找不是**有序表最好的查找方法。

例如:

关键字:	A	B	C	D	E
Pi:	0.2	0.3	0.05	0.3	0.15
Ci:	2	3	1	2	3

此时 ASL=2×0.2+3×0.3+1×0.05+2×0.3+3×0.1

若改变Ci的值

	2	1	3	2	3

则 ASL=2×0.2+1×0.3+3×0.05+2×0.3+3×0.1

定义:

使 $ASL = \sum_{i=1}^{n} p_i C_i + \sum_{i=1}^{n+1} q_i C_i'$

达最小的判定树称为最优二叉树，

其中: $\sum_{i=1}^{n} p_i + \sum_{i=1}^{n=1} q_i = 1$

介绍一种次优二叉树的构造方法：

为计算方便，令 $w_i = \alpha p_i$

选择二叉树的根结点，

使 $\Delta P_i = \left| \sum_{j=i+1}^{h} w_j - \sum_{j=1}^{i-1} w_j \right|$ 达最

为便于计算，引入累计权值和 $sw_i = \sum_{j=l}^{i} w_j$

并设 $w_{l-1} = 0$ 和 $sw_{l-1} = 0$，

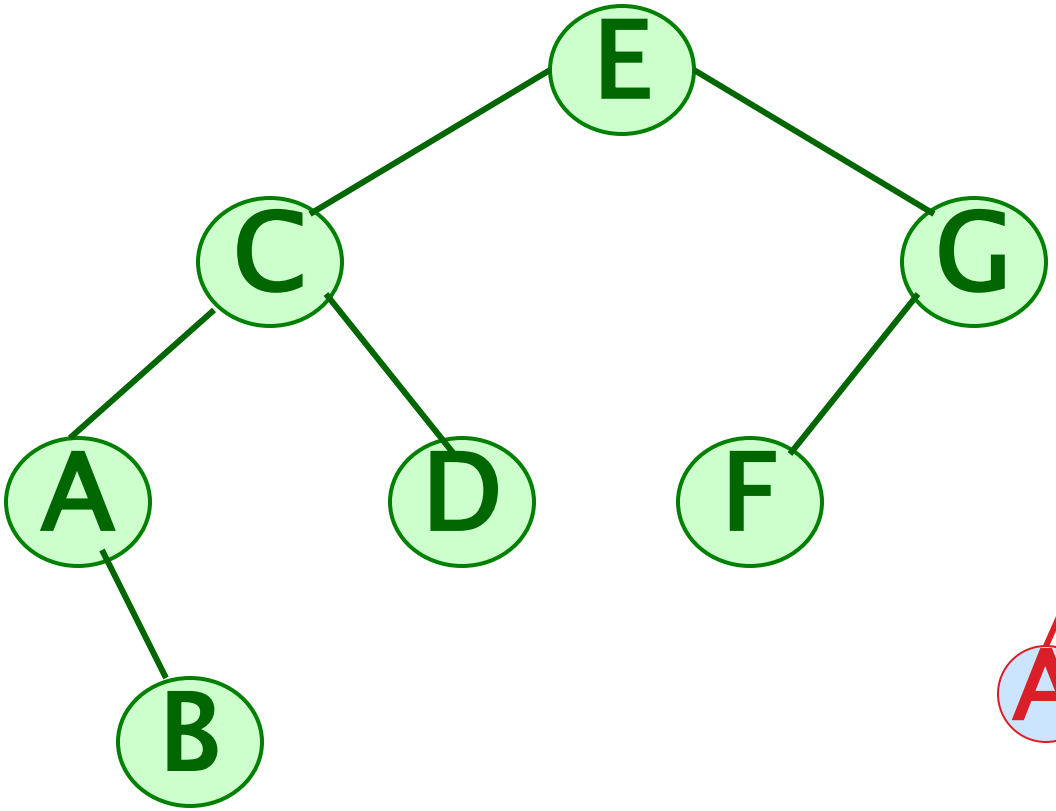
则推导可得 $\Delta P_i = \left| (sw_h + sw_{l-1}) - sw_i - sw_{i-1} \right|$

例如:

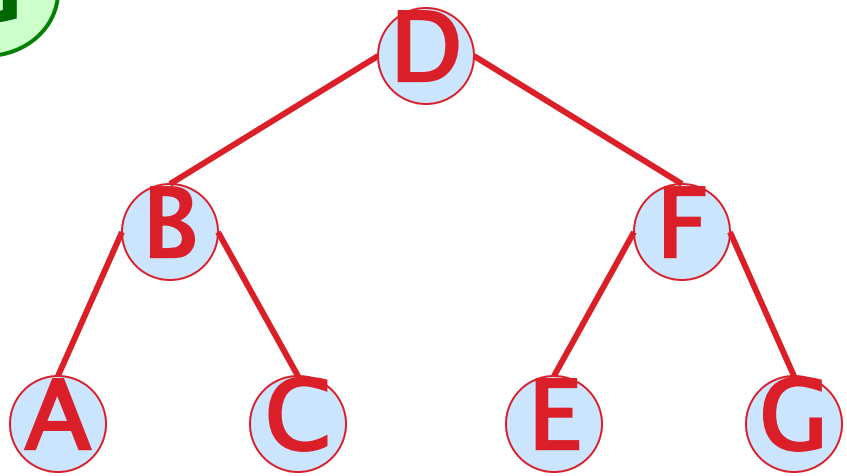
l *h*

j	0	1	2	3	4	5	6	7
w_j	0	2	1	5	3	4	3	5
sw_j	0	2	3	8	11	15	18	23
Δp_j		21	18	12	4	3	10	18
		9	6	0	8		5	3
		1	2					
key		A	B	C	D	E	F	G

所得次优二叉树如下所示:



和折半查找相比较



查找比较“总次数”
= 3×2+4×1+2×5+3×3 = 52

查找比较“总次数”
= 3×2+2×1+3×5+1×3
+3×4+2×3+3×5 = 5

构造次优二叉树的算法

```
Status SecondOptimal(BiTree &T, ElemType
                     float sw[], int low, int high) {
 // 由有序表R[low..high]及其累计权值表sw
 // 递归构造次优查找树T。
  选择最小的ΔP_i值 // 使根两侧的累计权值尽量接近
  if (!(T = (BiTree)malloc(sizeof(BiTNode
      return ERROR;
  T->data = R[i];                    // 生成结点
```

CONTINUE

```
if (i==low)  T->lchild = NULL;  // 左子树
else  SecondOptimal(T->lchild, R, sw, low,
                                    // 构造左子树
 if (i==high)  T->rchild = NULL;  // 右子
 else  SecondOptimal(T->rchild, R, sw, i+1,
                                    // 构造右子树
 return OK;
} // SecondOptimal
```

选择最小的ΔPi值

```
i=low;
min= abs(sw[high]-sw[low]);
dw = sw[high]+sw[low-1];

for (j=low+1; j<=high; ++j) //
    if abs(dw-sw[j]-sw[j-1]) < min) {
      i = j; min = abs(dw-sw[j]-sw[j-1]);
            } // if
```

次优查找树采用二叉链表的存储结构

```
Status CreateSOSTre(SOSTree &T, SSTable ST
  // 由有序表 ST 构造一棵次优查找树 T
  // ST 的数据元素含有权域 weight
  if (ST.length = 0)  T = NULL;
  else {
     FindSW(sw, ST);
                // 按照有序表 ST 中各数据元素
                // 的 weight 值求累计权值表
     SecondOpiamal(T, ST.elem, sw, 1, ST.leng
  }
  return OK;
} // CreatSOSTree
```



索引顺序表的查找过程：

1）由索引确定记录所在区间；

2）在顺序表的某个区间内进行查找。

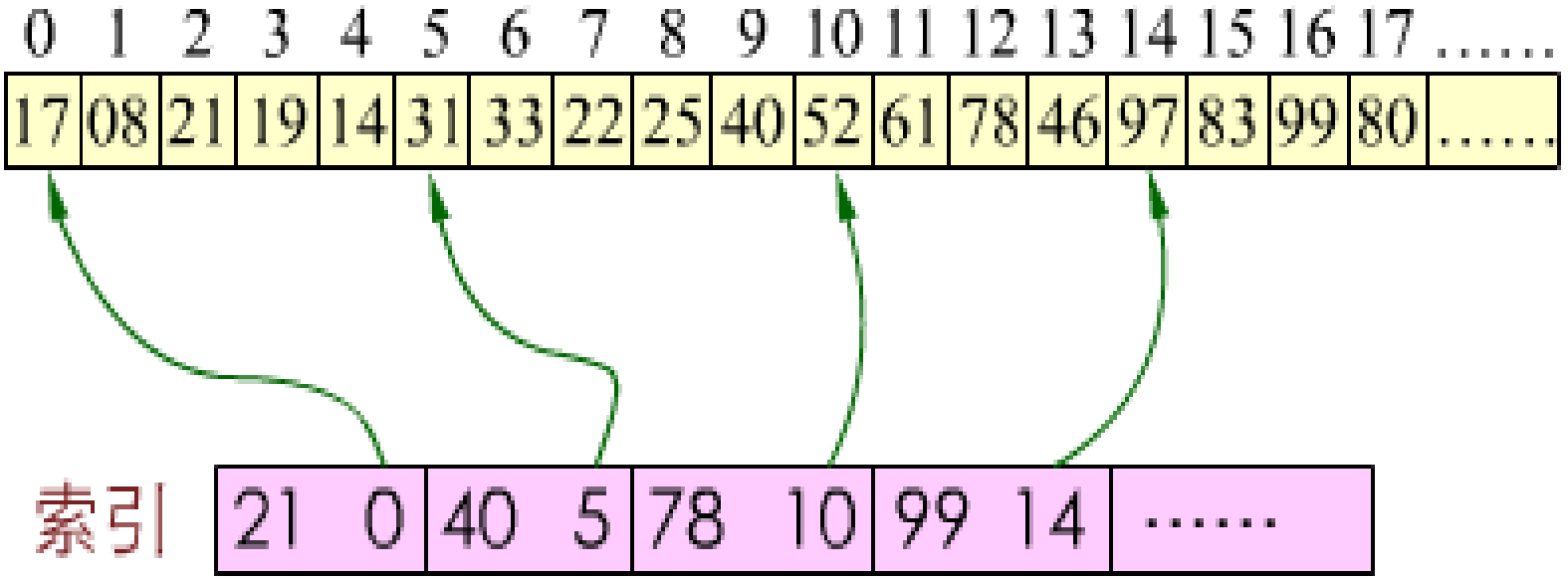
可见，

索引顺序查找的过程也是一个**“缩小区间”**的查找过程。

注意：索引可以根据查找表的特点来构造。

在建立顺序表的同时，建立一个“索引”。

例如：分块有序表



索引顺序表 = 索引 + 分块有序表

索引顺序查找的平均查找长度 =

查找“索引”的平均查找长度

＋ 查找“顺序表”的平均查找长度



抽象数据类型动态查找表的定义如下：

ADT DynamicSearchTable {

数据对象D： D是具有相同特性的数据元素的集合。
每个数据元素含有类型相同的关键字，可唯一标识数据元素。

数据关系R： 数据元素同属一个集合。

基本操作P：

InitDSTable(&DT)

DestroyDSTable(&DT)

SearchDSTable(DT, key);

InsertDSTable(&DT, e);

DeleteDSTable(&T, key);

TraverseDSTable(DT, Visit())

}ADT DynamicSearchTable



InitDSTable(&DT);

操作结果：

构造一个空的动态查找表DT。



DestroyDSTable(&DT);

初始条件：动态查找表DT存在；

操作结果：销毁动态查找表DT。



SearchDSTable(DT, key);

初始条件： 动态查找表DT存在，**key**为和关键字类型相同的给定值；

操作结果： 若DT中存在其关键字等于**key**的数据元素，则函数值为该元素的值或在表中的位置，否则为“空”。



InsertDSTable(&DT, e);

初始条件：动态查找表DT存在，
e 为待插入的数据元素；

操作结果：若DT中不存在其关键字
等于 e.key 的 数据元素，
则插入 e 到DT。



DeleteDSTable(&T, key);

初始条件：动态查找表DT存在，key为和关键字类型相同的给定值；

操作结果：若DT中存在其关键字等于key的数据元素，则删除之。



TraverseDSTable(DT, Visit());

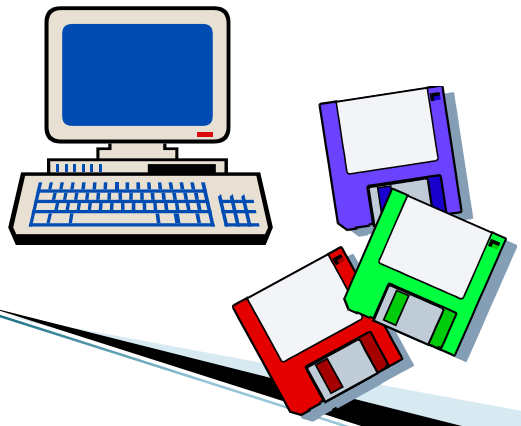
初始条件： 动态查找表DT存在，**Visit** 是对结点操作的应用函数；

操作结果： 按某种次序对DT的每个结点调用函数 **Visit()** 一次且至多一次。一旦 **Visit()** 失败，则操作失败。



9.2

动态查找树表



综合上一节讨论的几种查找表的特性：

	查找	插入	删除
无序顺序表	O(n)	O(1)	O(n)
无序线性链表	O(n)	O(1)	O(1)
有序顺序表	O(logn)	O(n)	O(n)
有序线性链表	O(n)	O(1)	O(1)
静态查找树表	O(logn)	O(nlogn)	O(nlogn)

可得如下结论：

1）从查找性能看，最好情况能达O(logn)，此时要求表有序；

2）从插入和删除的性能看，最好情况能达O(1)，此时要求存储结构是链表。

一、二叉排序树（二叉查找树）

二、二叉平衡树

三、B－树

四、B^+树

五、键　树



一、二叉排序树

（二叉查找树）

1．定义

2．查找算法

3．插入算法

4．删除算法

5．查找性能的分析



1．定义：

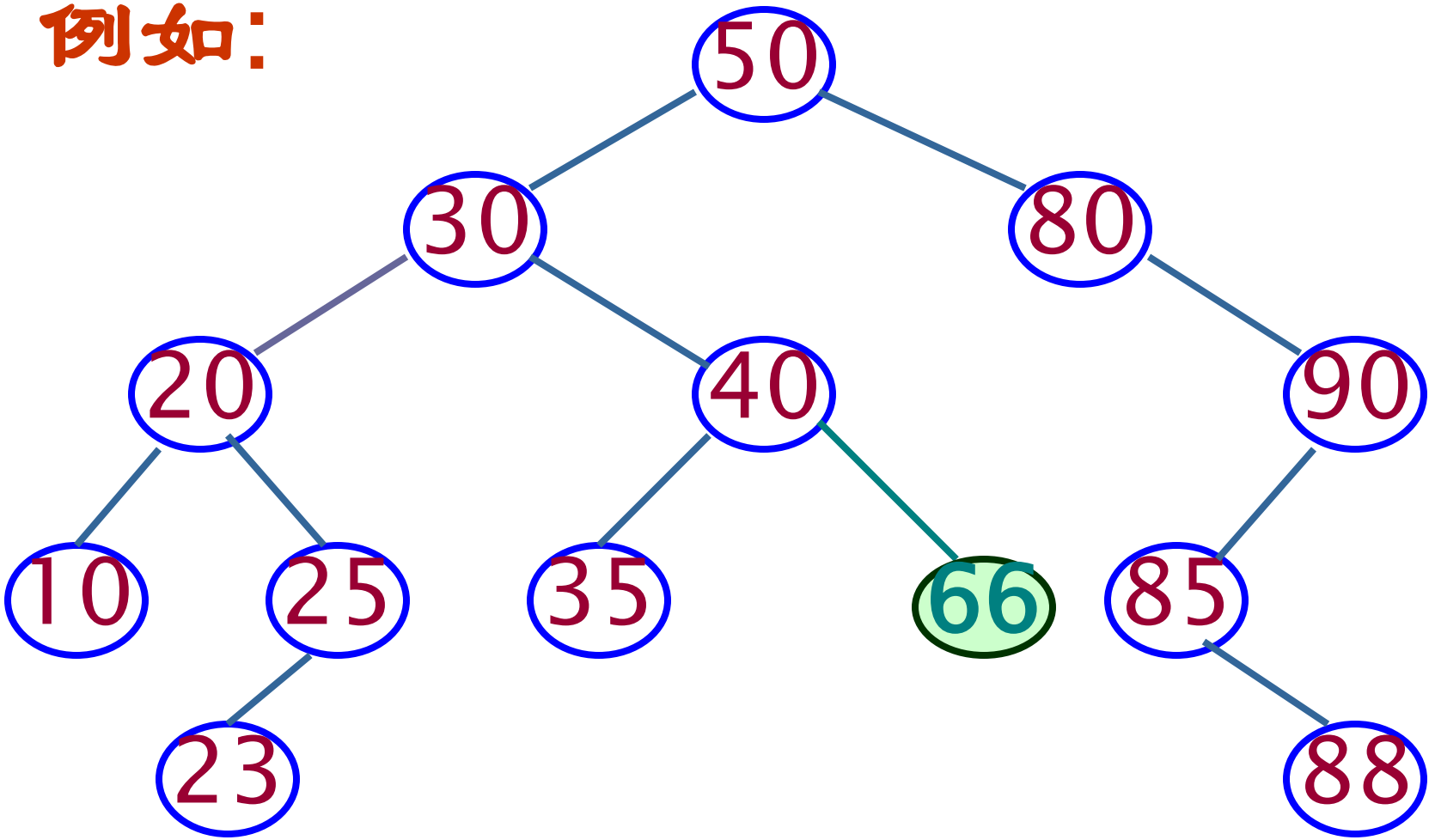
二叉排序树或者是一棵空树；或者是具有如下特性的二叉树：

（1）若它的左子树不空，则左子树上所有结点的值均小于根结点的值；

（2）若它的右子树不空，则右子树上所有结点的值均大于根结点的值；

（3）它的左、右子树也都分别是二叉排序树。

例如：



不是二叉排序树。

通常，取二叉链表作为二叉排序树的存储结构

```
typedef struct BiTNode { // 结点结构
TElemType      data;

    struct BiTNode  *lchild, *rchild;
                                  // 左右孩子指针
} BiTNode, *BiTree;
```



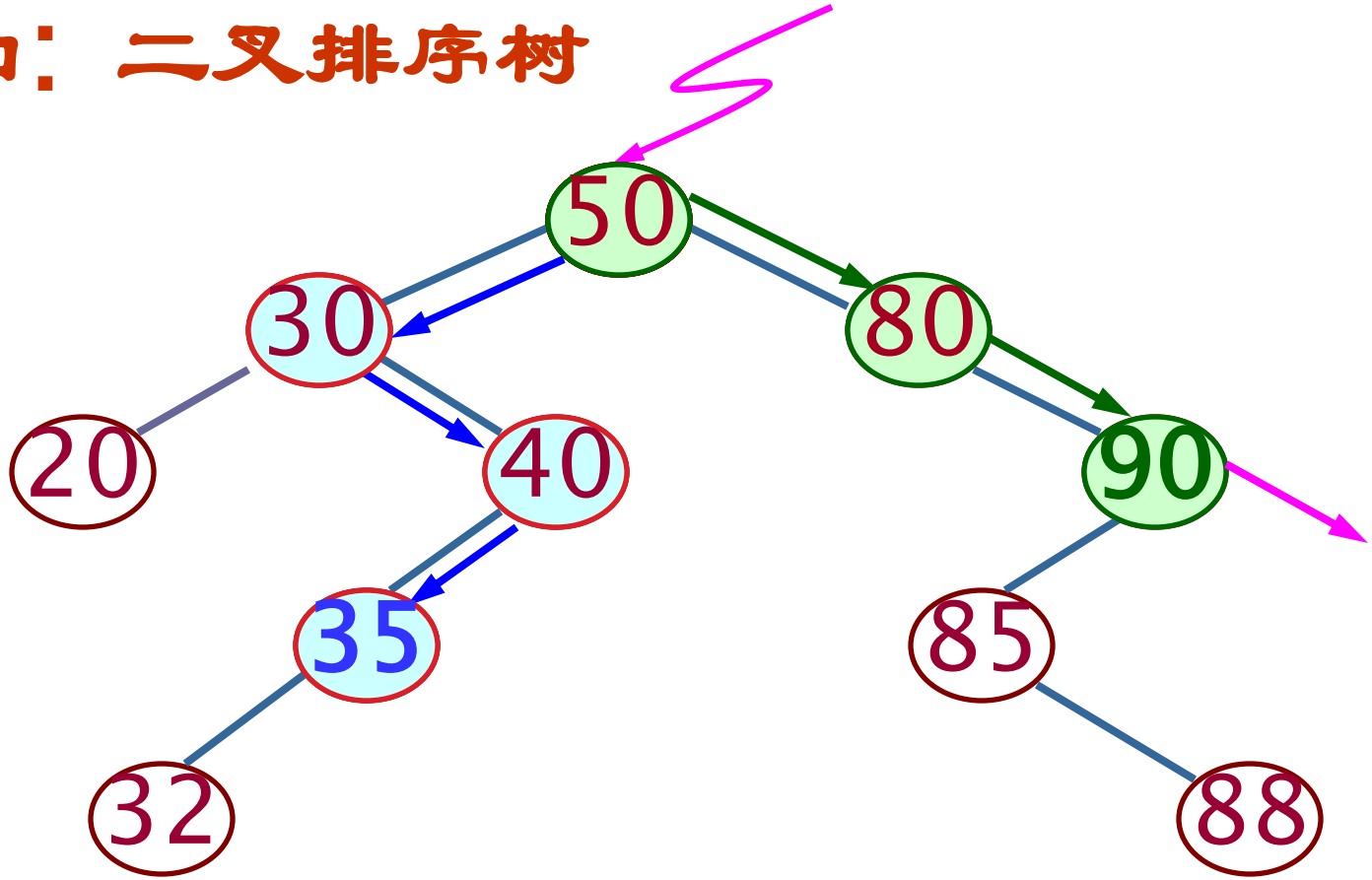
2．二叉排序树的查找算法：

若二叉排序树**为空**，则**查找不成功**；

否则，

- 1）若给定值**等于**根结点的关键字，则**查找成功**；
- 2）若给定值**小于**根结点的关键字，则**继续在左子树上进行查找**；
- 3）若给定值**大于**根结点的关键字，则**继续在右子树上进行查找。**

例如：二叉排序树



查找关键字

== 50,35,90,95,

从上述查找过程可见，

在查找过程中，生成了一条**查找路径**：

从根结点出发，沿着左分支或右分支逐层向下直至关键字等于给定值的结点；

——查找成功

或者

从根结点出发，沿着左分支或右分支逐层向下直至指针指向空树为止。

——查找不成功

算法描述如下：

```
Status SearchBST (BiTree T, KeyType key,
                                    BiTree f, BiTree &p ) {
  // 在根指针 T 所指二叉排序树中递归地查找其
  // 关键字等于 key 的数据元素，若查找成功，则返回指针 p 指向查找路径上访问的最后一个结点，并返回
  // 函数值为 TRUE；
```

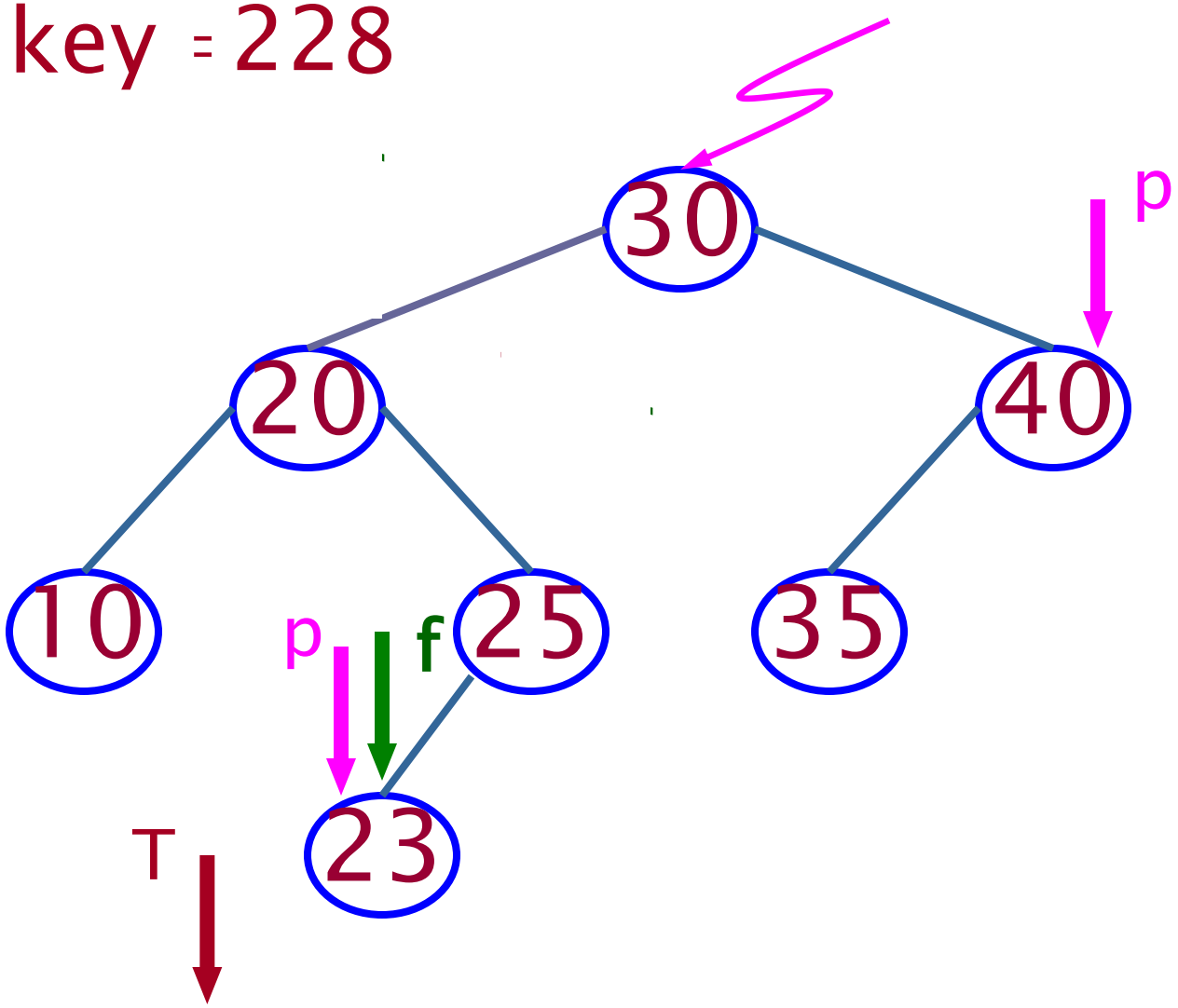
否则表明查找不成功，指针 p 指向查找路径上访问的最后一个结点，并返回函数值为FALSE，指针 f 指向当前访问的结点的双亲，其初始调用值为NULL

... ...

```
if (!T) // 空子树表示查找失败
    { p = f;  return FALSE; }  // 查找不成功
else  if ( EQ(key, T->data.key) )
    { p = T;  return TRUE; }  // 查找成功
else  if ( LT(key, T->data.key) )
     SearchBST (T->lchild, key, T, p );
                          // 在左子树中继续查找
elseSearchBST (T->rchild, key, T, p );
                          // 在右子树中继续查找
```



设 key = 228



3．二叉排序树的插入算法

- 根据动态查找表的定义，“**插入**”**操作在查找不成功时才进行**；

- 若二叉排序树为**空树**，则新插入的结点为**新的根结点**；否则，新插入的结点必为一个**新的叶子结点**，其**插入位置**由查找过程得到。

```
Status InsertBST(BiTree &T, ElemType e)
{
  // 当二叉排序树中不存在关键字等于 e.key 的
  // 数据元素时，插入元素值为 e 的结点，并返
  // 回 TRUE; 否则，不进行插入并返回FALSE
   if (!SearchBST ( T, e.key, NULL, p ))
     {                                  }
   else return FALSE;
} // InsertBST
```



```
s = (BiTree) malloc (sizeof (BiTNode));
                        // 为新结点分配空间
s->data = e;
s->lchild = s->rchild = NULL;
if ( !p ) T = s;      // 插入 s 为新的根结点
else   if ( LT(e.key, p->data.key) )
       p->lchild = s;          // 插入 *s 为 *p 的左
else  p->rchild = s;      // 插入 *s 为 *p 的
return TRUE;      // 插入成功
```



4．二叉排序树的删除算法

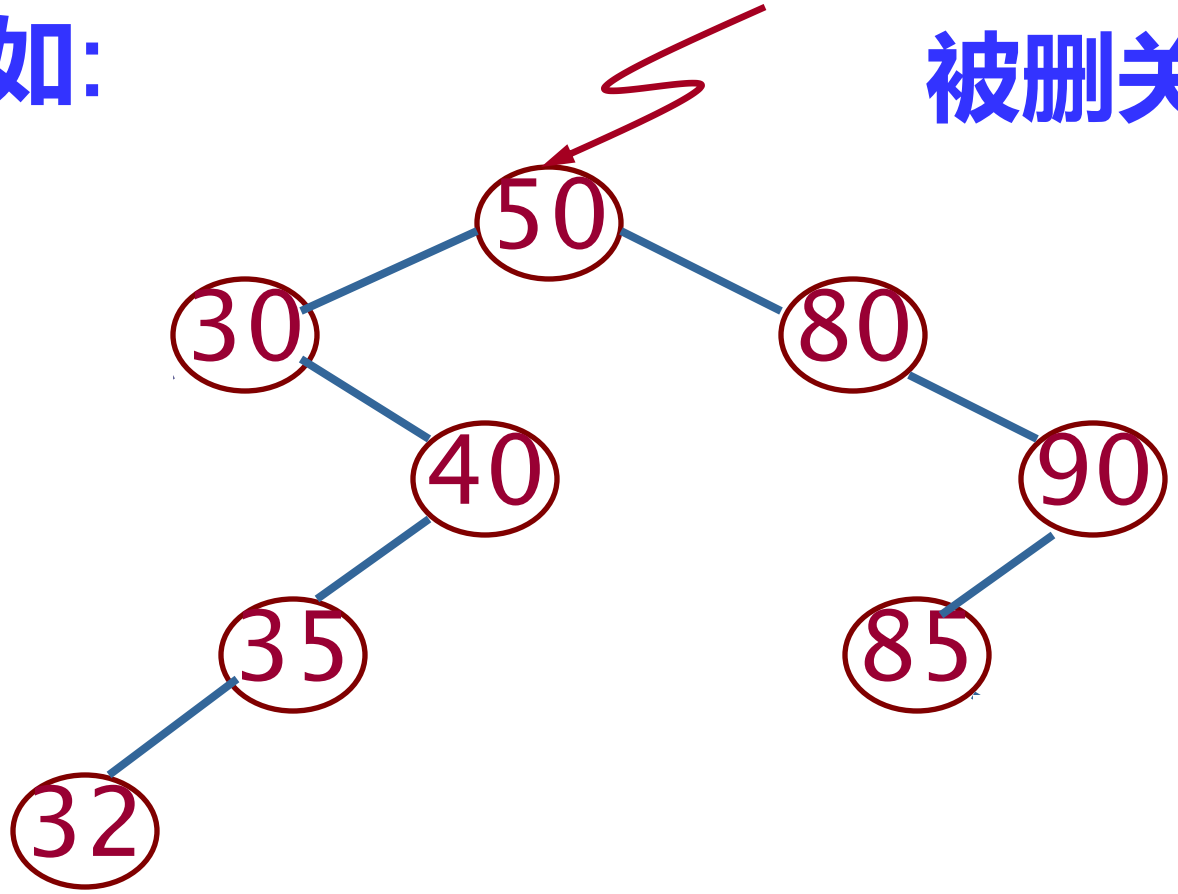
和插入相反，删除在**查找成功**之后进行，并且要求在删除二叉排序树上某个结点之后，**仍然保持二叉排序树的特性。**

可分三种情况讨论：

- （1）被删除的结点是叶子；
- （2）被删除的结点只有左子树或者只有右子树；
- （3）被删除的结点既有左子树，也有右子树。

（1）被删除的结点是叶子结点

例如: **被删关键字 = 88)**

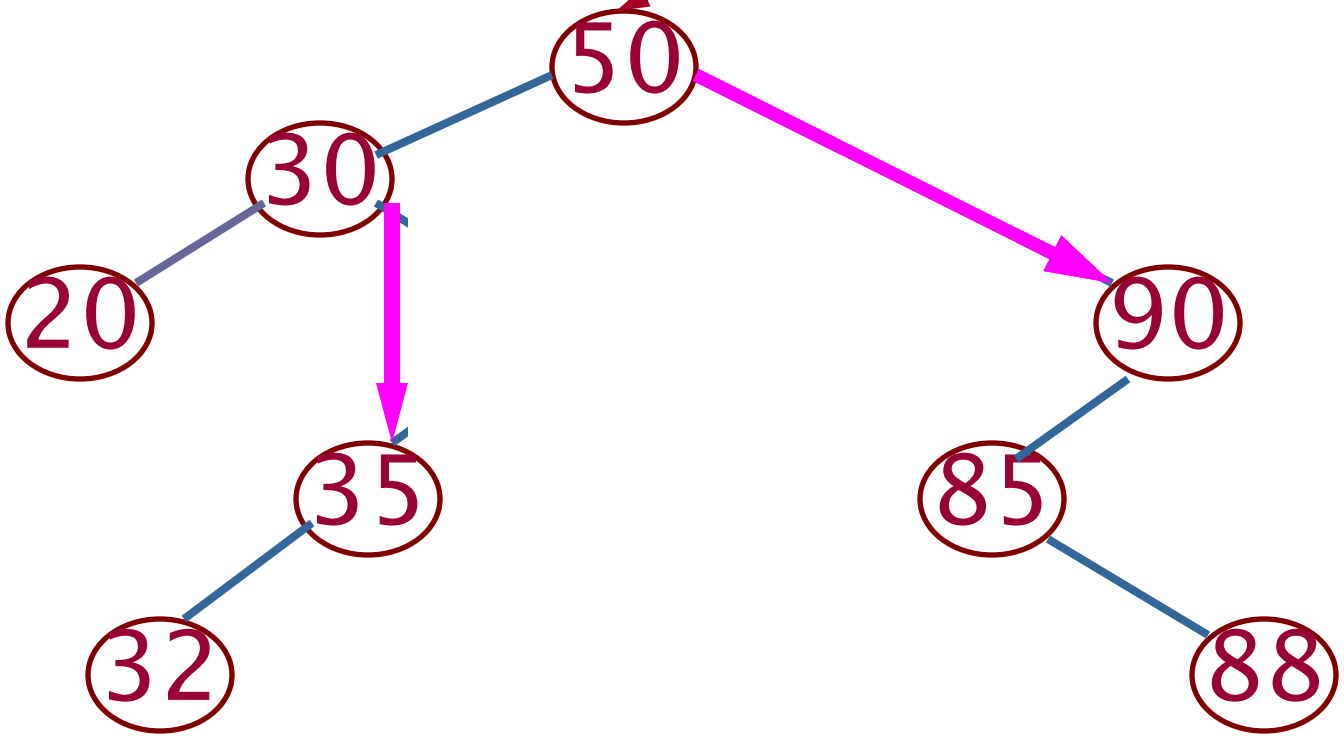


其双亲结点中相应指针域的值改为“空”

（2）被删除的结点只有左子树

或者只有右子树

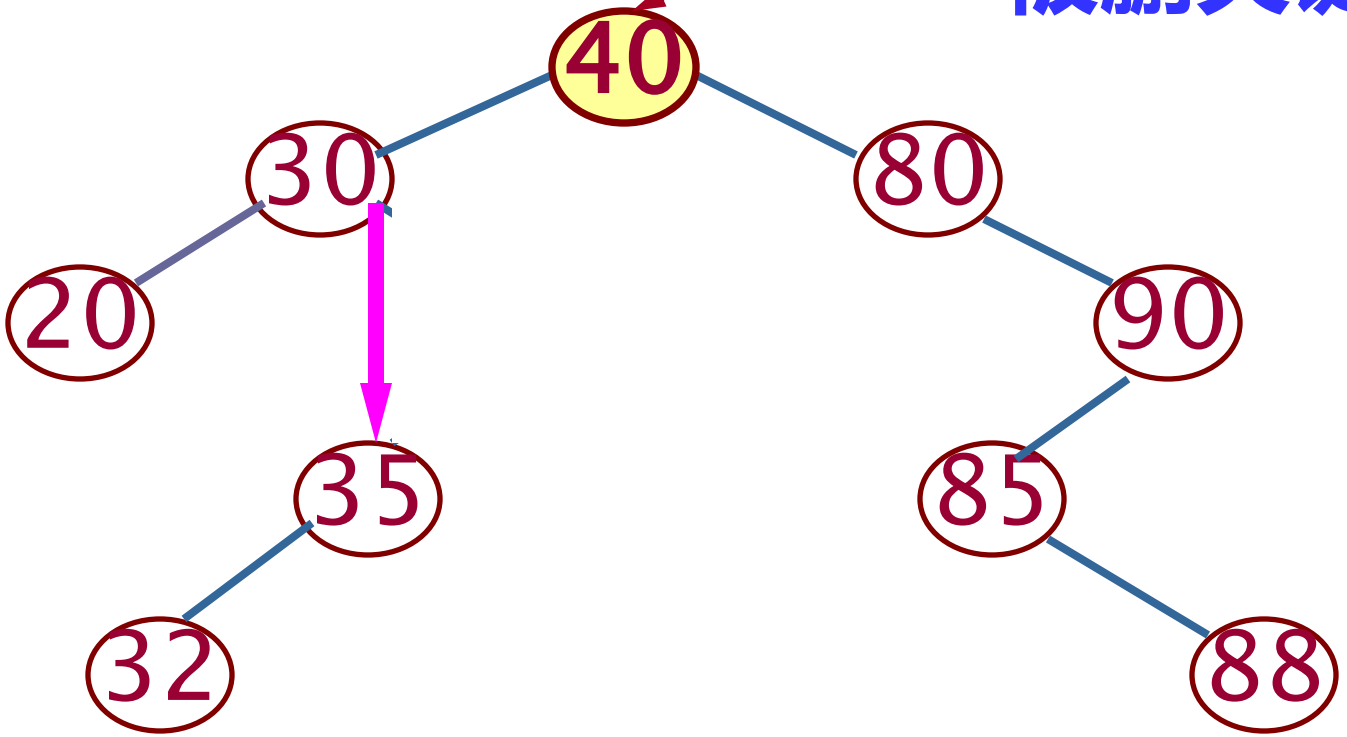
被删关键字 = 80

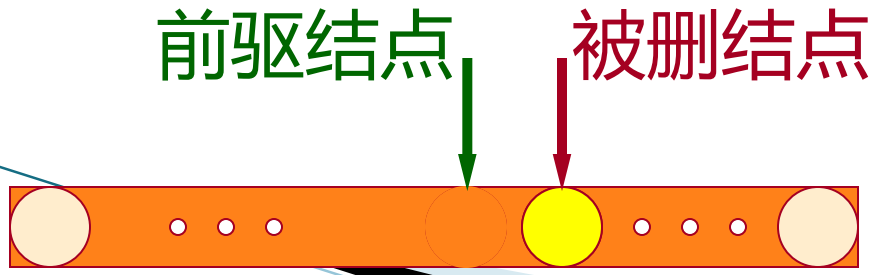


其双亲结点的相应指针域的值改为“指向被删除结点的左子树或右子树”。

（3）被删除的结点既有左子树，也有右子树

被删关键字 = 50





以其前驱替代之，然后再删除该前驱结点

算法描述如下：

```
Status DeleteBST (BiTree &T，KeyType key
  // 若二叉排序树 T 中存在其关键字等于 key 的
  // 数据元素，则删除该数据元素结点，并返回
  // 函数值 TRUE，否则返回函数值 FALSE
  if (!T)  return FALSE;
       // 不存在关键字等于key的数据元素
  else {     … …          }
} // DeleteBST
```



```
if ( EQ (key, T->data.key) )
   {  Delete (T);   return TRUE;  }
        // 找到关键字等于key的数据元素
else if ( LT (key, T->data.key) )
        DeleteBST ( T->lchild, key );
             // 继续在左子树中进行查找
else DeleteBST ( T->rchild, key );
             // 继续在右子树中进行查找
```



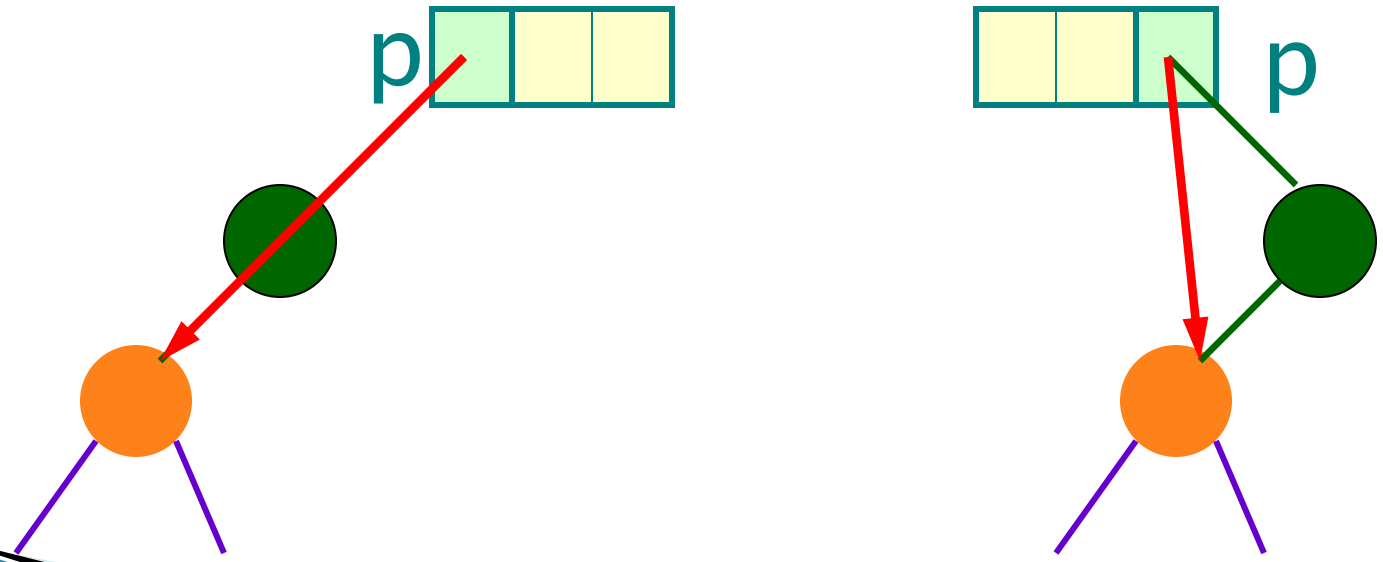
其中**删除操作**过程如下所描述：

```
void Delete ( BiTree &p ){
  // 从二叉排序树中删除结点 p，
  // 并重接它的左子树或右子树
  if (!p->rchild) .{. ...          }
  else if (!p->lchild) .{ ...          }
  else {... ...          } // 双子树情形用中序前驱替换
} // Delete
```



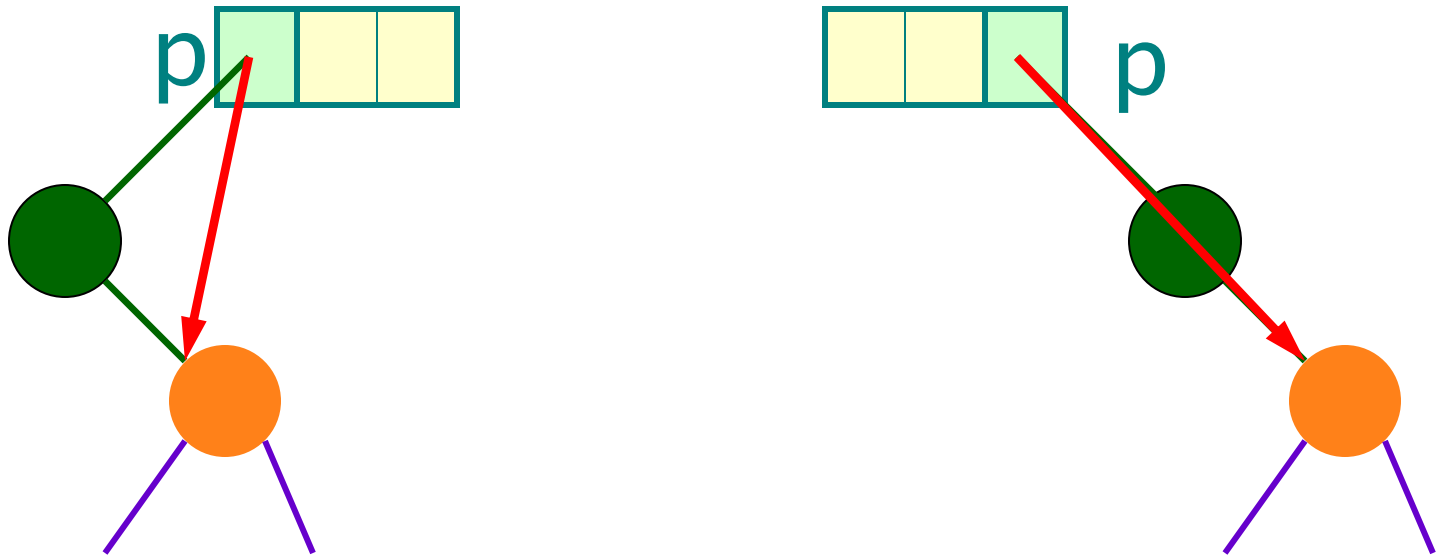
// 右子树为空树则只需重接它的左子树

q = p; p = p->lchild; free(q);

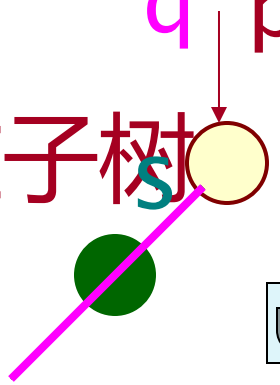


// 左子树为空树只需重接它的右子树

q = p; p = p->rchild; free(q);



```
        // 左右子树均不空
q = p;  s = p->lchild;
while (!s->rchild) { q = s;  s = s->rchil
                        // s 指向被删结点的前驱
p->data = s->data;
if (q != p )  q->rchild = s->lchild;
else  q->lchild = s->lchild;          q  p
                        // 重接*q的左子树
free(s);
```

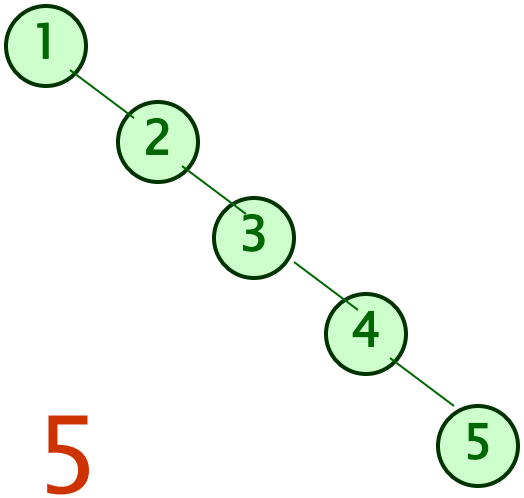


5．查找性能的分析

对于每一棵特定的二叉排序树，均可按照平均查找长度的定义来求它的 *ASL* 值，显然，由值相同的 *n* 个关键字，构造所得的不同形态的各棵二叉排序树的平均查找长 度的值不同，甚至可能差别很大。

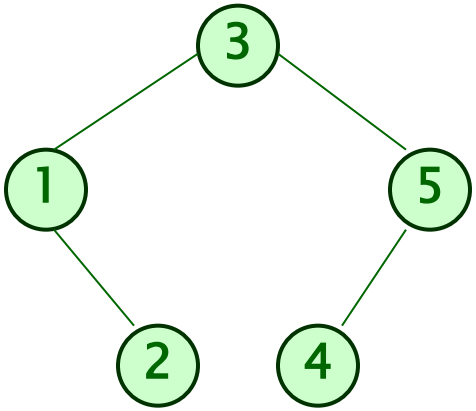
例如：

由关键字序列 1，2，3，4，5构造而得的二叉排序树，



ASL = (1+2+3+4+5) / 5 = 3

由关键字序列 3，1，2，5，4构造而得的二叉排序树，

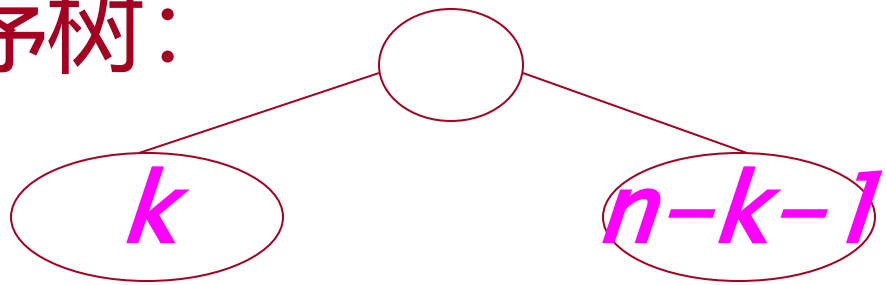


ASL = (1+2+3+2+3) / 5 = 2.2

二叉排序树的查找性能取决于二叉排序树的形状，在O(log2n)和O(n)之间。

下面讨论平均情况：

不失一般性，假设长度为 *n* 的序列中有 *k* 个关键字**小于**第一个关键字，则必有 *n-k-1* 个关键字**大于**第一个关键字,由它构造的二叉排序树：



的平均查找长度是 *n* 和 *k* 的函数

$P(n, k)$　　$(0 \le k \le n-1)$。

假设 n 个关键字可能出现的 $n!$ 种排列的可能性相同，则含 n 个关键字的二叉排序树的平均查找长度：

$$ASL = P(n) = \frac{1}{n}\sum_{k=0}^{n-1} P(n,k)$$

在**等概率查找**的情况下，

$$P(n,k) = \sum_{i=1}^{n} p_i C_i = \frac{1}{n}\sum_{i=1}^{n} C_i$$

$$P(n,k) = \frac{1}{n}\sum_{i=1}^{n} C_i = \frac{1}{n}\left(C_{root} + \sum_{L} C_i + \sum_{R} C_i \right)$$

$$= \frac{1}{n}\left(1 + k(P(k)+1) + (n-k-1)(P(n-k-1)+1)\right)$$

$$= 1 + \frac{1}{n}\left(k \times P(k) + (n-k-1) \times P(n-k-1)\right)$$

由此

$$P(n)=\frac{1}{n}\sum_{k=0}^{n-1}\left(1+\frac{1}{n}\left(k\times P(k)+(n-k-1)\times P(n-k-1)\right)\right)$$

$$=1+\frac{2}{n^2}\sum_{k=1}^{n-1}\left(k\times P(k)\right)$$

可类似于解差分方程，此递归方程有解：

$$P(n)=2\frac{n+1}{n}\log n+C$$



二、二叉平衡树

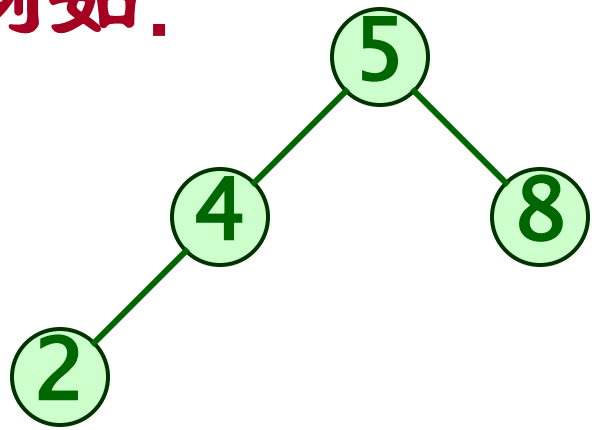
- 何谓“二叉平衡树”？
- 如何构造“二叉平衡树”
- 二叉平衡树的查找性能分析



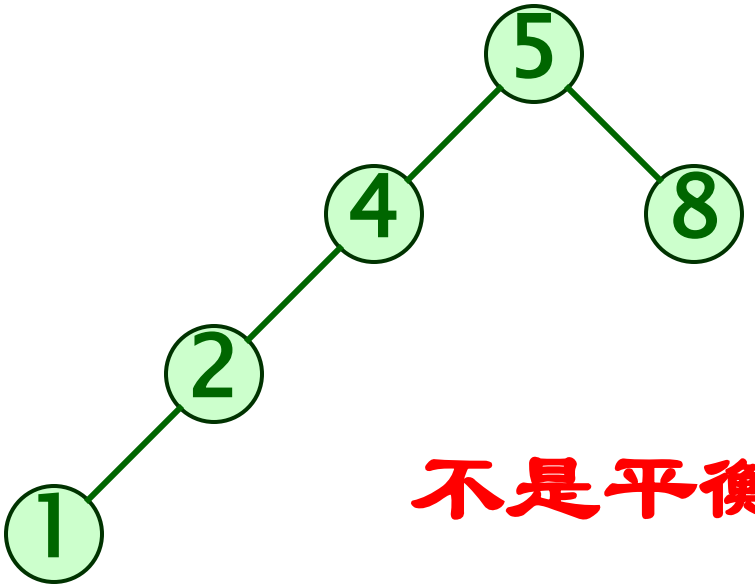
二叉平衡树是二叉查找树的另一种形式，其特点为：

树中每个结点的左、右子树深度之差的绝对值不大于1 $|h_L - h_R| \leq 1$ 。

例如：



是平衡树

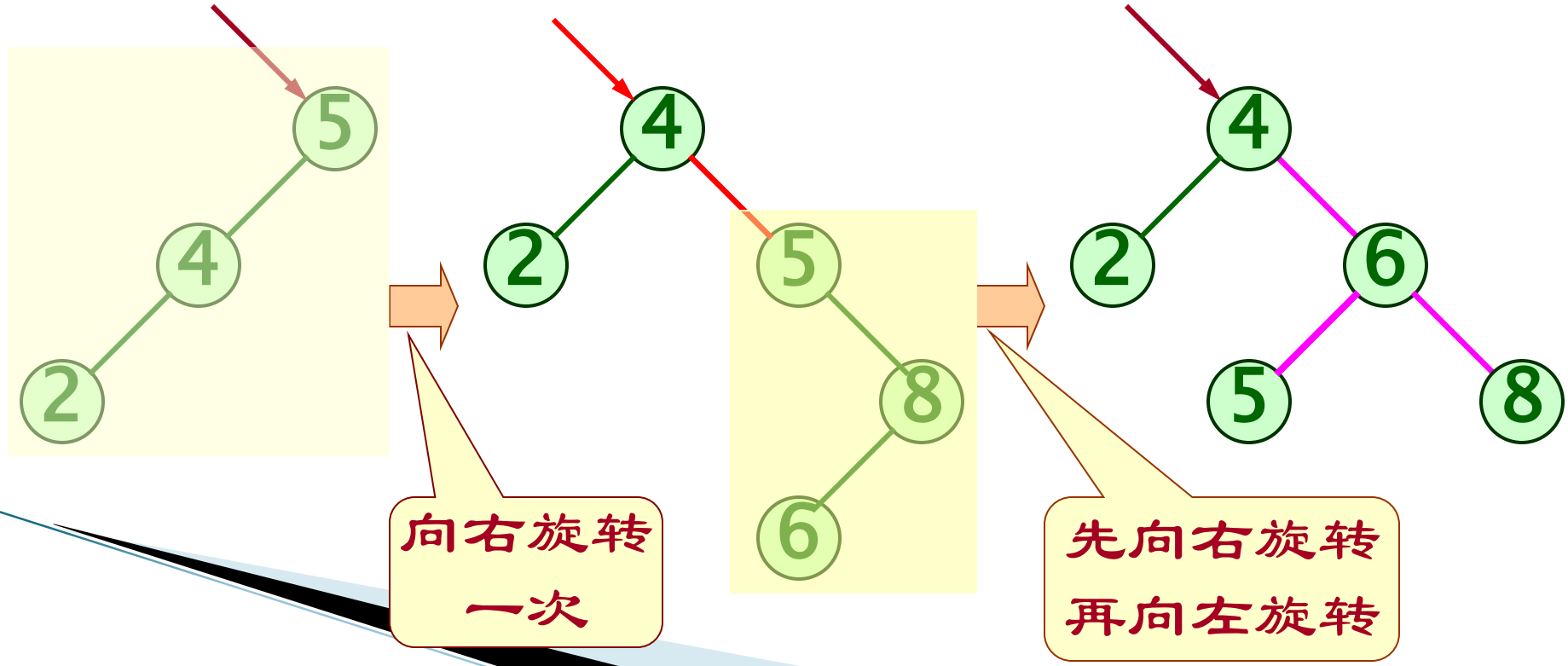


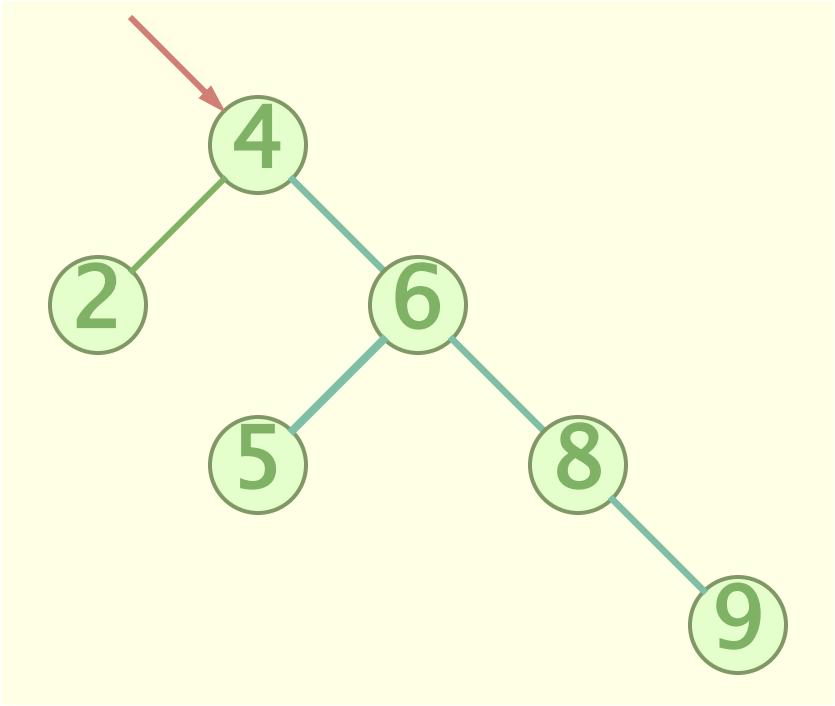
不是平衡树

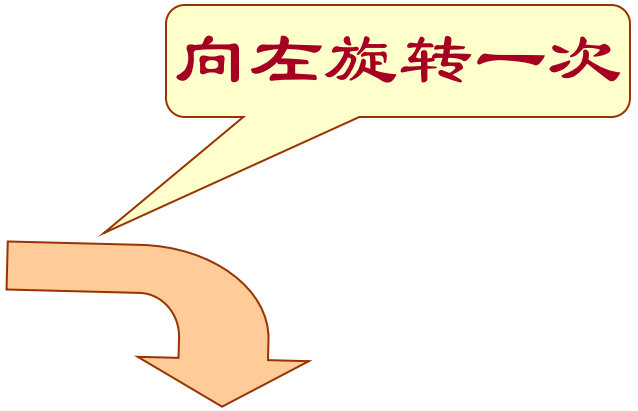
构造二叉平衡（查找）树的方法是：

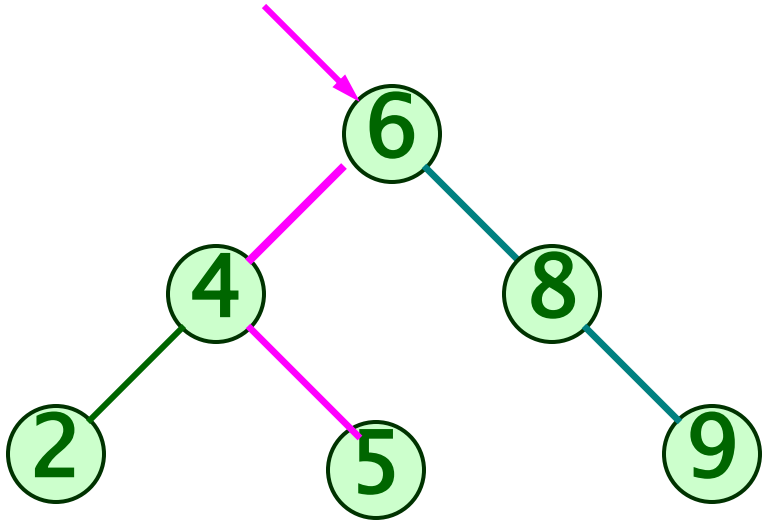
在插入过程中，采用平衡旋转技术。

例如:依次插入的关键字为5, 4, 2, 8, 6, 9





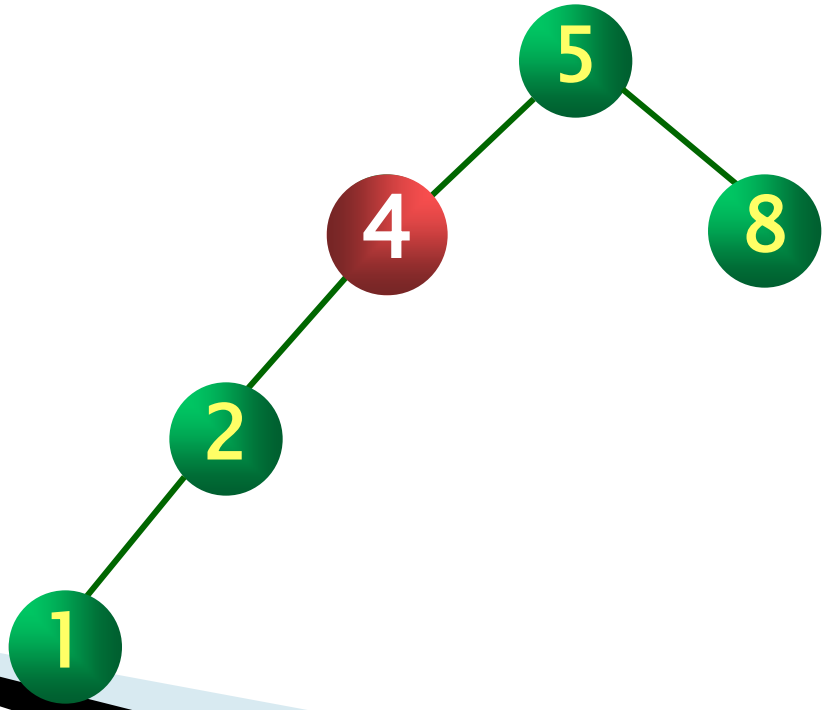




继续插入关键字 9

平衡二叉树

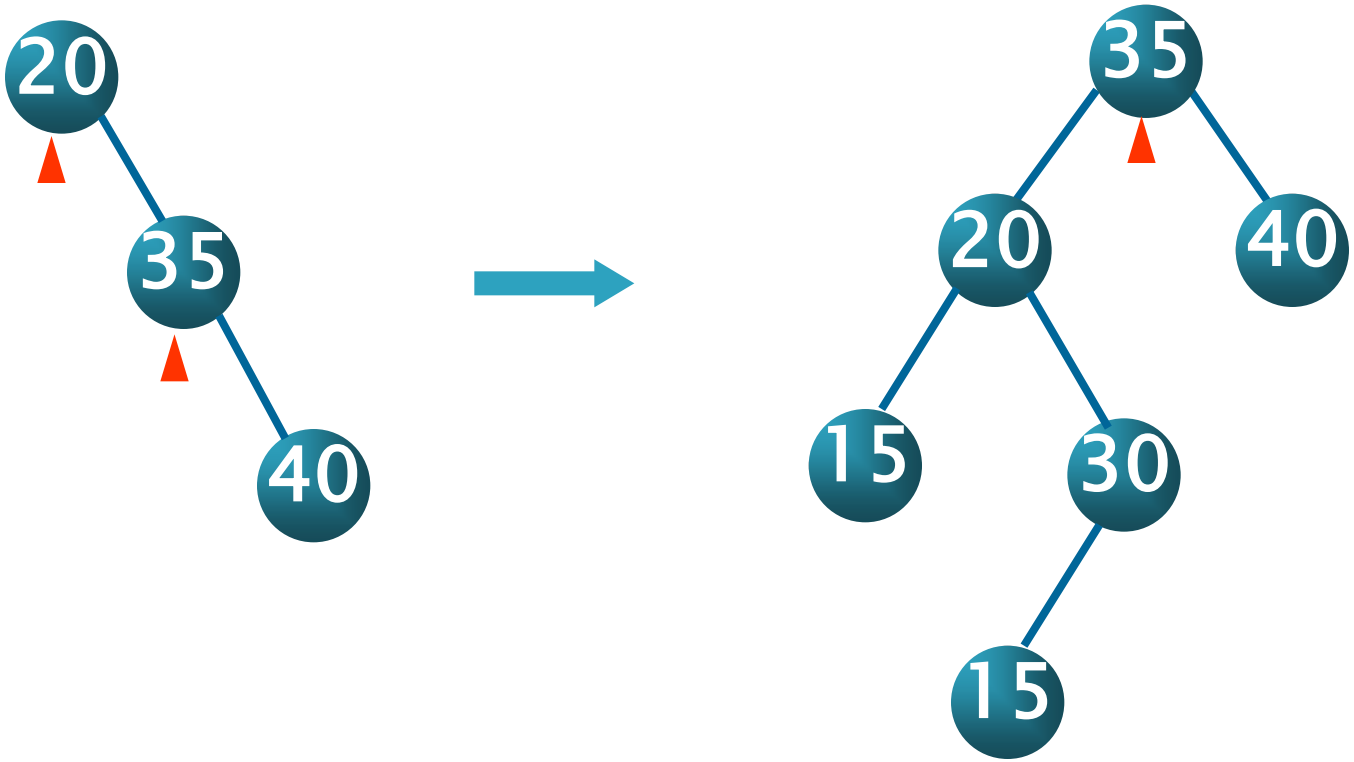
最小不平衡子树：在平衡二叉树的构造过程中，以距离**插入结点**最近的、且平衡因子的绝对值大于1的结点为**根**的子树。



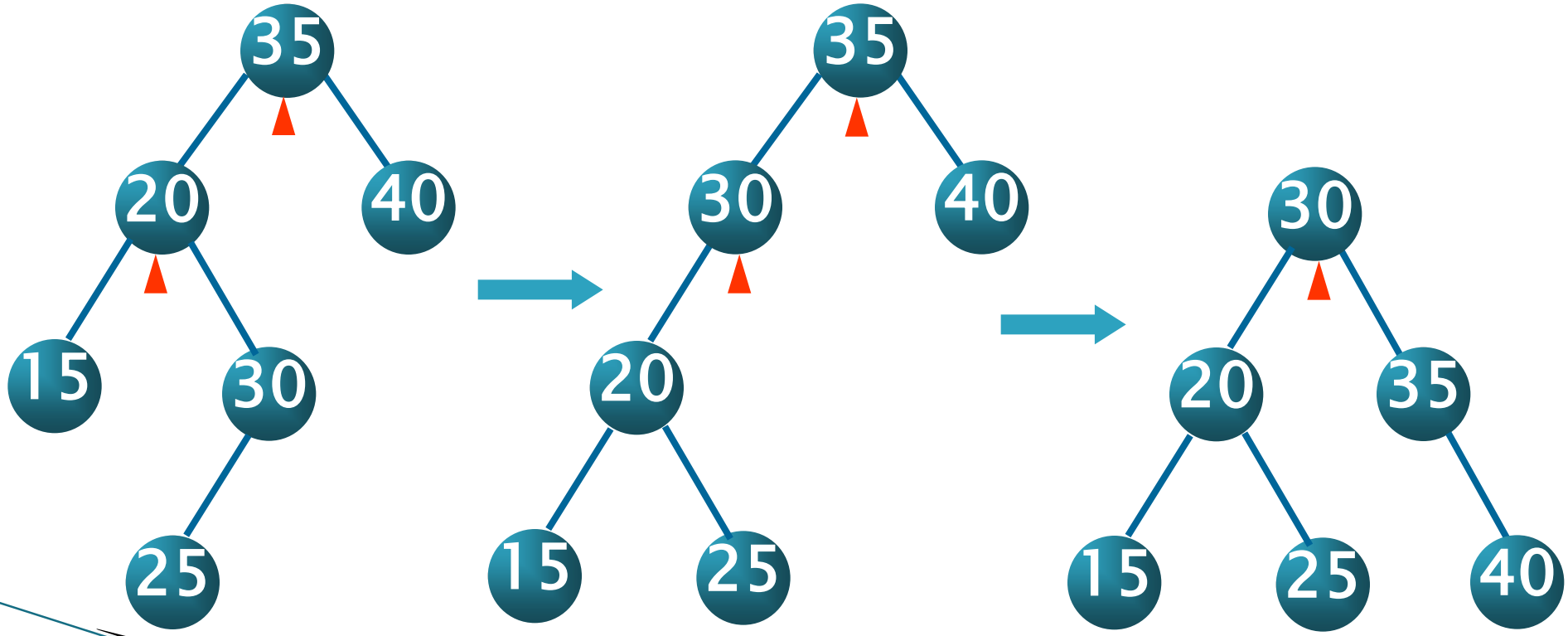
平衡二叉树

基本思想：在构造二叉排序树的过程中，每插入一个结点时，首先检查是否因插入而破坏了树的平衡性，若是，则找出最小不平衡子树，在保持二叉排序树特性的前提下，调整最小不平衡子树中各结点之间的链接关系，进行相应的旋转，使之成为新的平衡子树。

例：设序列{20，35，40，15，30，25}，构造平衡树。



例：设序列{20，35，40，15，30，25}，构造平衡树。

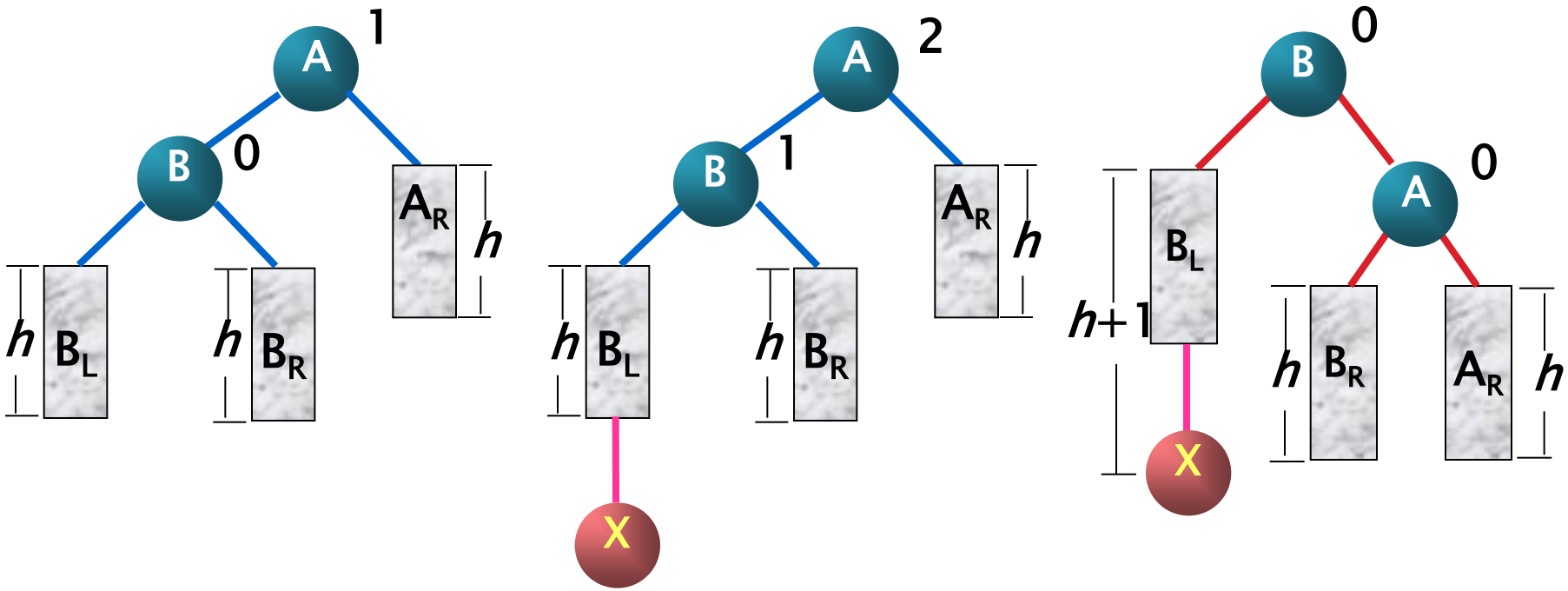


平衡二叉树

设结点A为最小不平衡子树的根结点，对该子树进行平衡调整归纳起来有以下四种情况：

1. LL型
2. RR型
3. LR型
4. RL型

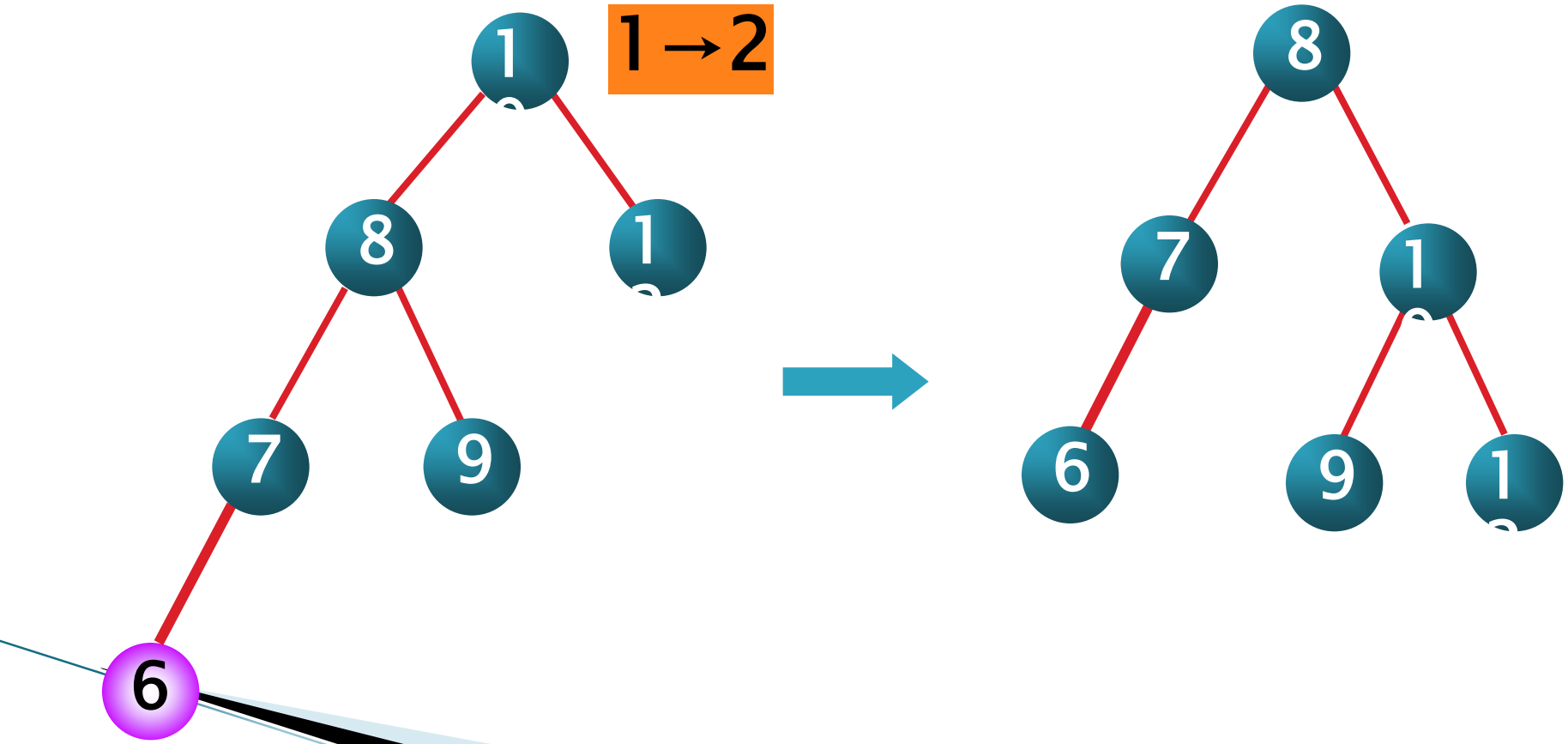
平衡二叉树——LL型



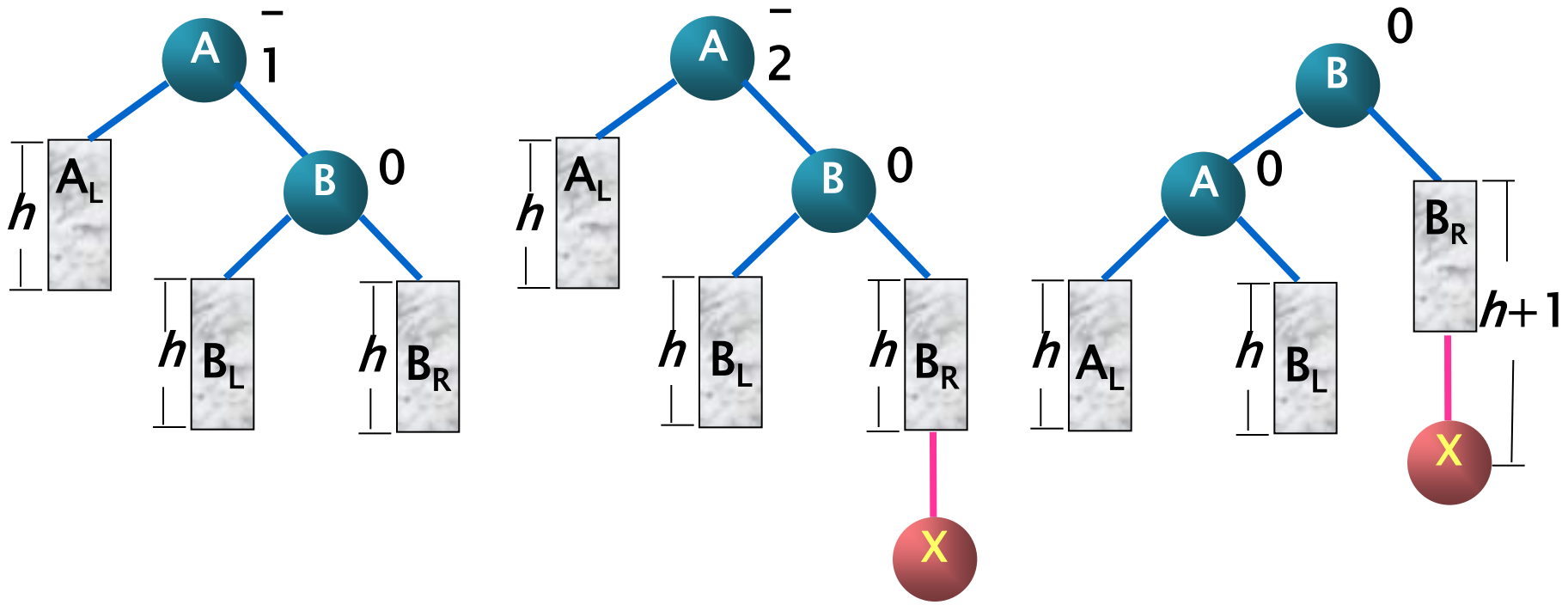
插入前　　插入后，调整前　　调整后

旋转：扁担原理；冲突：旋转优先

例:LL型

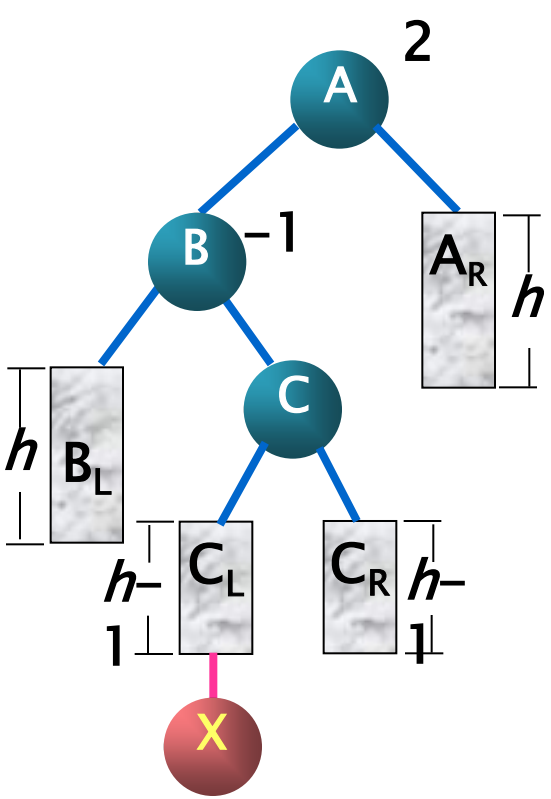


平衡二叉树——RR型

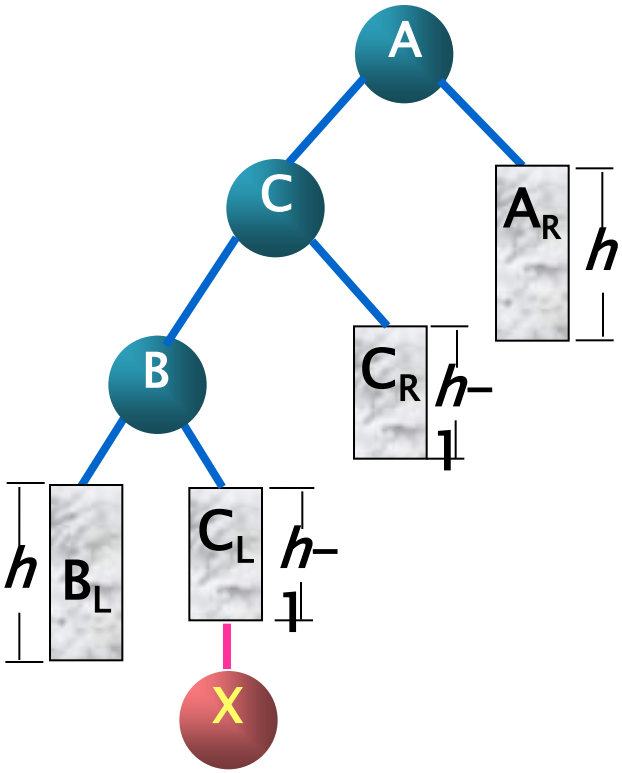


插入前　　插入后，调整前　　调整后

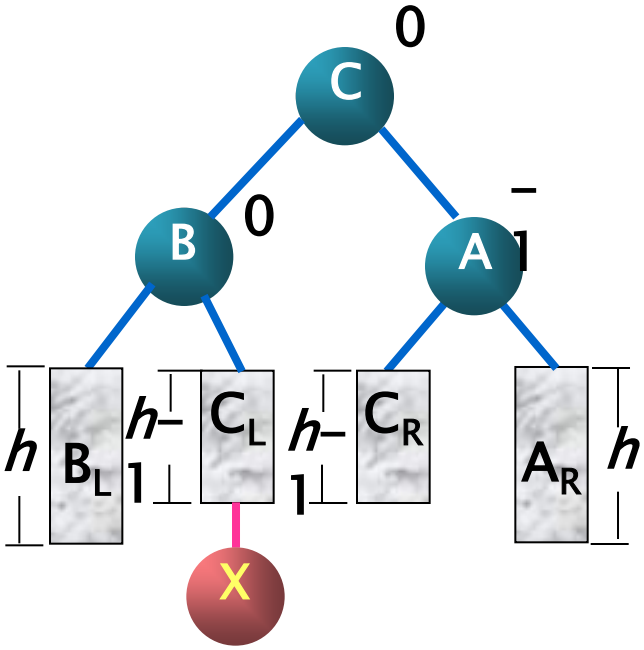
平衡二叉树——LR型



插入后，调整前

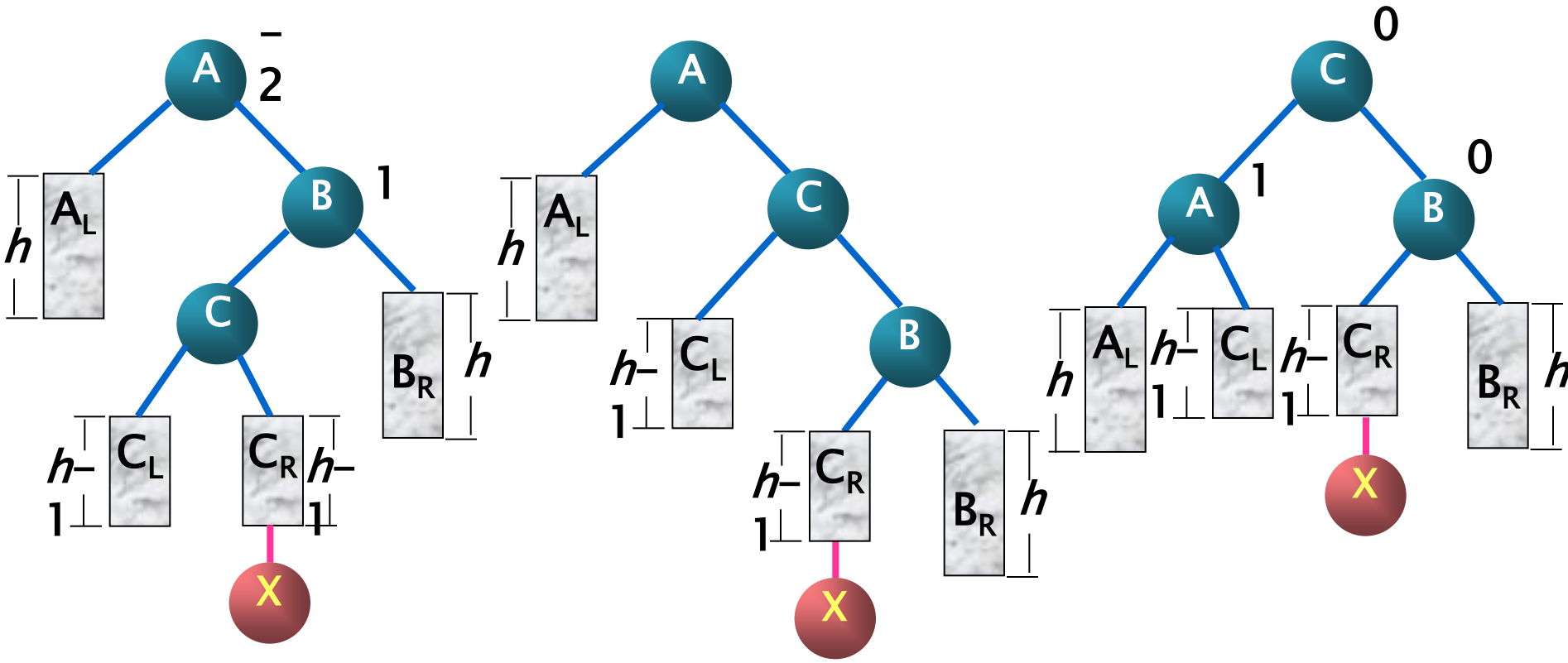


先顺时针旋转



再逆时针旋转

平衡二叉树——RL型



插入后，调整前　　先顺时针旋转　　再逆时针旋转

二叉平衡树的四种旋转

第 93–98 页已有 LL、RR、LR、RL 的分类与图示。补充指针旋转和子树根更新的代码。平衡因子沿用左子树高度减右子树高度。插入结点后，从下往上调用 rebalance，并用返回值重新连接子树根。

```
typedef struct AvlNode {
    int key, height;
    struct AvlNode *left, *right;
} AvlNode;

static int height(const AvlNode *p)
{
    return p ? p->height : 0;
}

static void update_height(AvlNode *p)
{
    int left = height(p->left), right = height(p->right);
    p->height = 1 + (left > right ? left : right);
}

static int balance(const AvlNode *p)
{
    return height(p->left) - height(p->right);
}
```

二叉平衡树的四种旋转

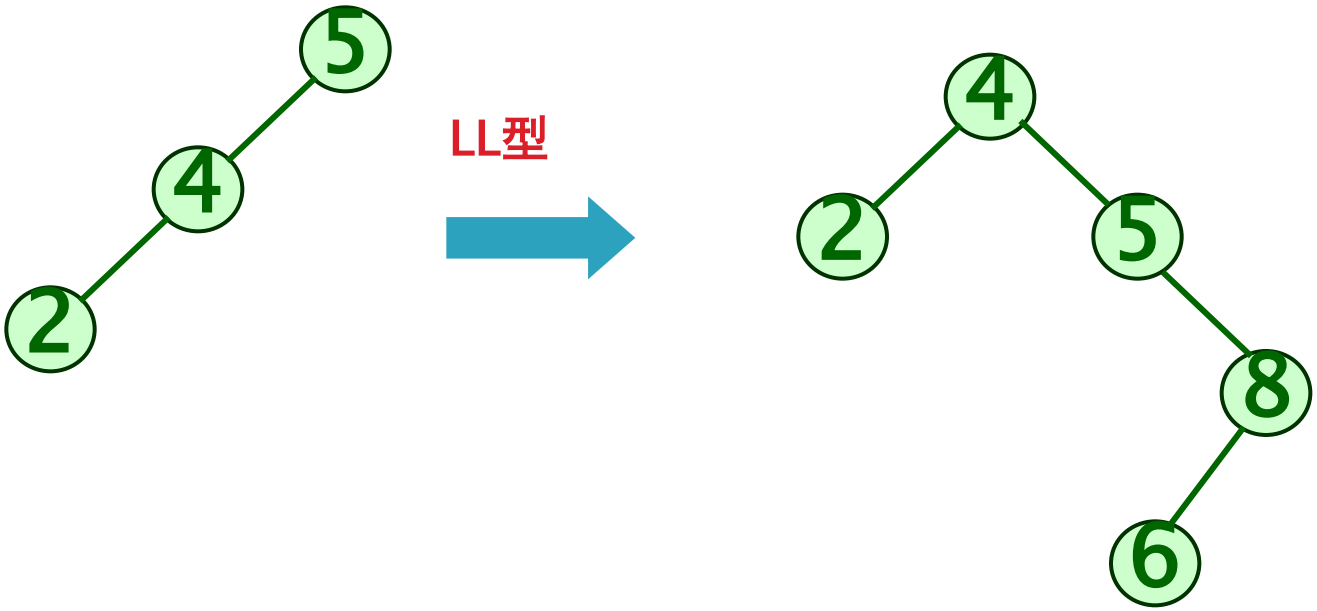
```
static AvlNode *rotate_right(AvlNode *a)
{
    AvlNode *b = a->left;
    a->left = b->right;  // 中间子树仍在 a 与 b 之间
    b->right = a;
    update_height(a);
    update_height(b);
    return b;
}

static AvlNode *rotate_left(AvlNode *a)
{
    AvlNode *b = a->right;
    a->right = b->left;
    b->left = a;
    update_height(a);
    update_height(b);
    return b;
}
```

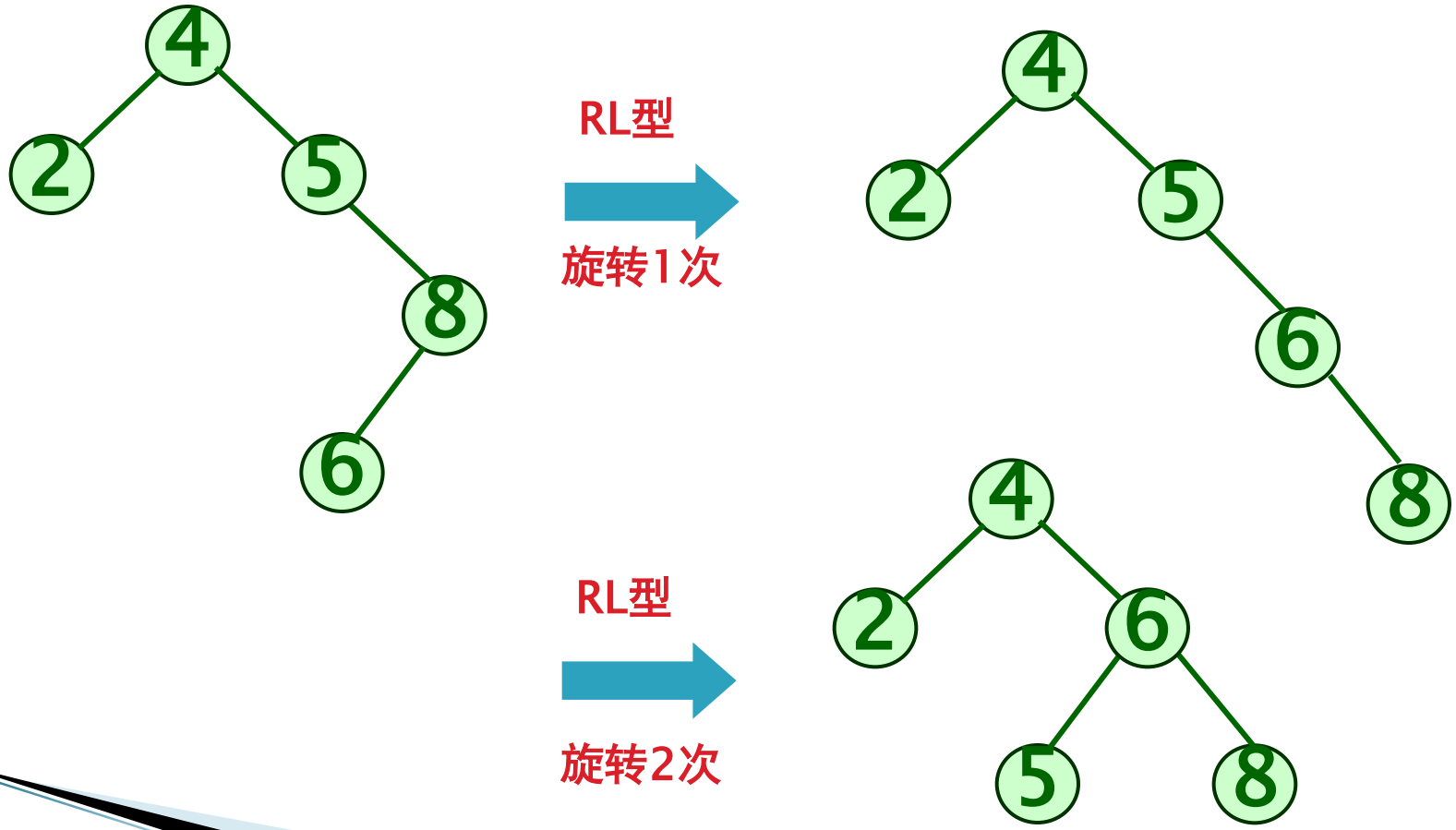
二叉平衡树的四种旋转

```
AvlNode *rebalance(AvlNode *a)
{
    update_height(a);
    if (balance(a) > 1) {
        if (balance(a->left) < 0) {  // LR 先旋转左孩子
            a->left = rotate_left(a->left);
        }
        return rotate_right(a);     // LL 直接右旋
    }
    if (balance(a) < -1) {
        if (balance(a->right) > 0) {  // RL 先旋转右孩子
            a->right = rotate_right(a->right);
        }
        return rotate_left(a);       // RR 直接左旋
    }
    return a;
}
```

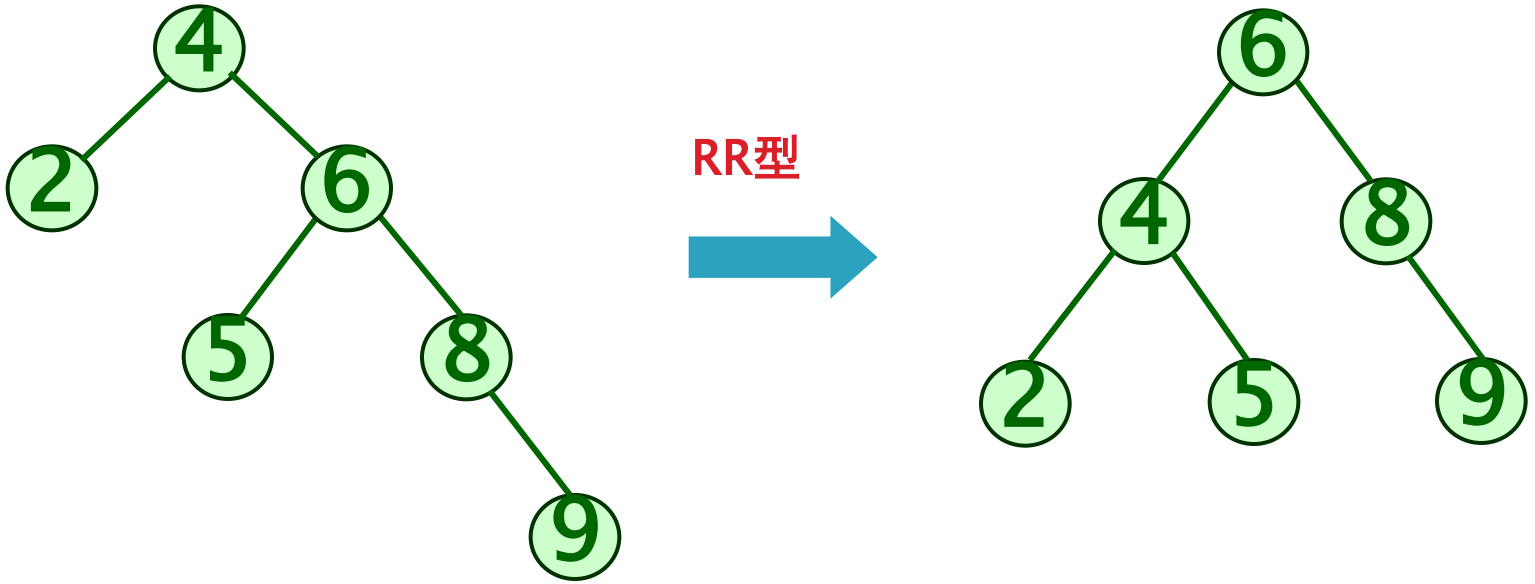
设有关键码序列{5, 4, 2, 8, 6, 9}，构造平衡树



设有关键码序列{5, 4, 2, 8, 6, 9}，构造平衡树



设有关键码序列{5, 4, 2, 8, 6, 9}，构造平衡树



平衡树的查找性能分析：

在平衡树上进行查找的过程和二叉排序树相同，因此，查找过程中和给定值**进行比较的关键字的个数**不超过平衡 树的深度。

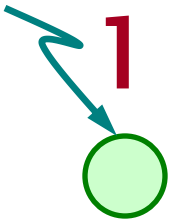
问：含 ***n*** 个关键字的二叉平衡树可能达到的**最大深度**是多少？

先看几个具体情况：

$n=0$
空树

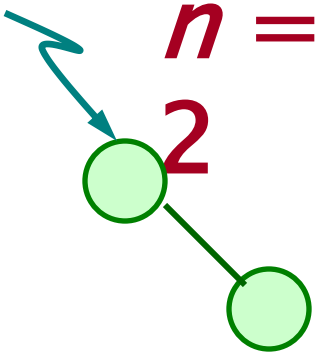
最大深度为0

$n=1$



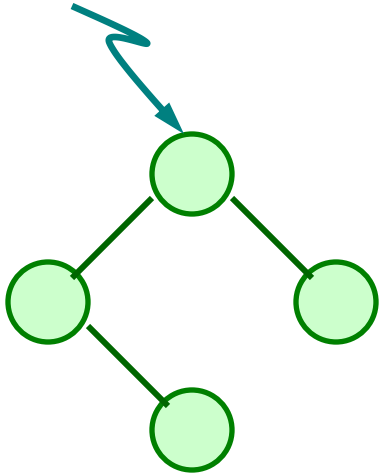
最大深度为 1

$n=2$



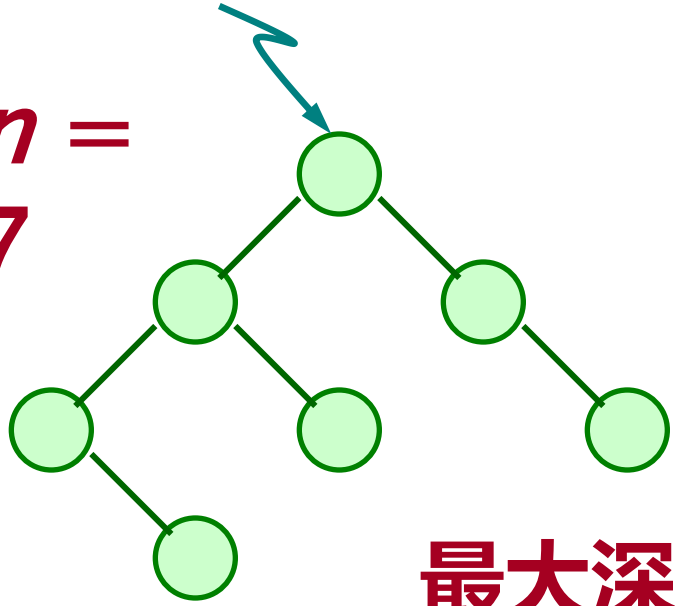
最大深度为 2

$n=4$



最大深度为3

$n=7$



最大深度为4

反过来问，**深度为 h 的二叉平衡树**中所**含结点的最小值 N_h** 是多少?

$h = 0 \quad N_0 = 0$　　$h = 1 \quad N_1 = 1$

$h = 2 \quad N_2 = 2$　　$h = 3 \quad N_3 = 4$

一般情况下　　$N_h = N_{h-1} + N_{h-2} + 1$

利用归纳法可证得　　$N_h = F_{h+2} - 1$

由此推得，**深度为 *h* 的二叉平衡树中所含结点的最小值** $N_h = \varphi^{h+2}/5 - 1$。反之，**含有 *n* 个结点的二叉平衡树能达到的最大深度** $h_n = \log_\varphi(\sqrt{5}(n+1)) - 2$。因此，在**二叉平衡树**上进行查找时，查找过程中和给定值**进行比较的关键字的个数和 *log(n)*** 相当。

三、 B－树

1．定义

2．查找过程

3．插入操作

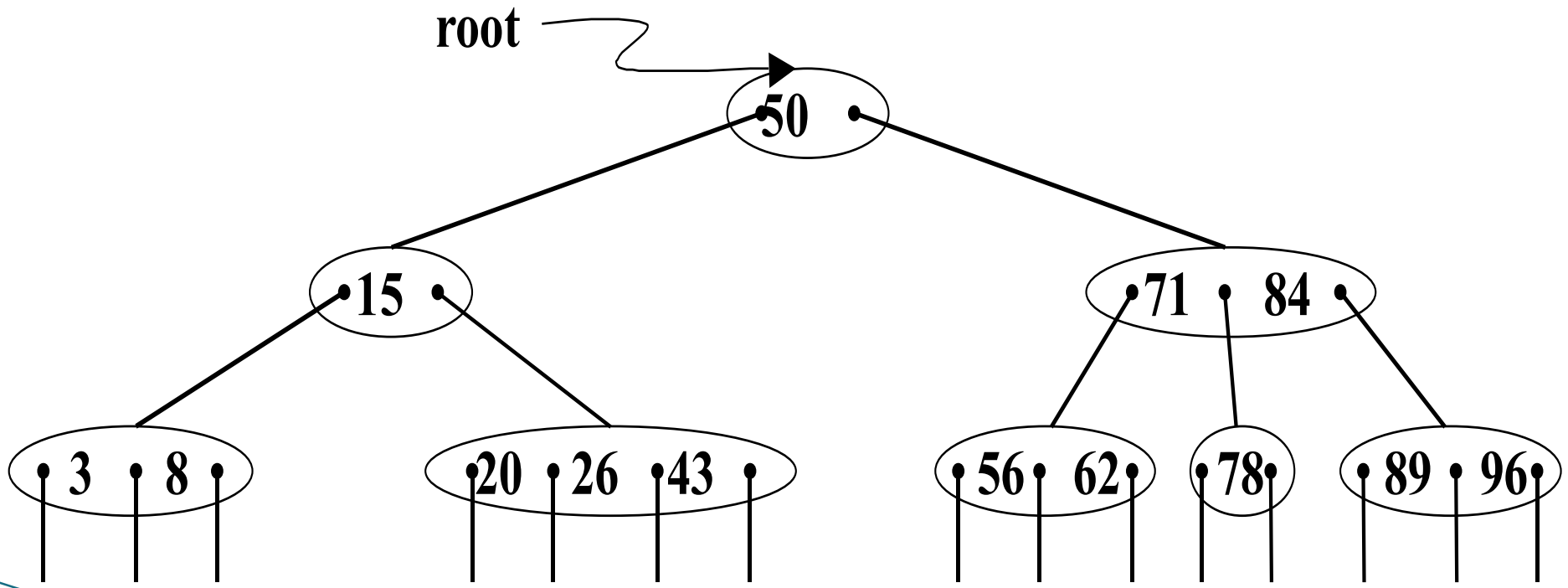
4．删除操作

5．查找性能的分析



1．B-树的定义

B-树是一种 平衡 的 多路 查找 树:



在 m 阶的B-树上，每个非终端结点可能含有：

n 个**关键字** K_i（$1 \le i \le n$）　$n < m$

n 个**指向记录的指针** D_i（$1 \le i \le n$）

$n+1$ 个**指向子树的指针** A_i（$0 \le i \le n$）

——多叉树的特性

B-树结构的C语言描述如下：

```
typedef struct BTNode {
  int  keynum;          // 结点中关键字个数，结点
  struct BTNode  *parent;
                        // 指向双亲结点的指针
  KeyType   key[m+1]; // 关键字（0号单元不用
  struct BTNode  *ptr[m+1]; // 子树指针向量
  Record        *recptr[m+1];  // 记录指针向量
} BTNode, *BTree; // B树结点和B树的类型
```

- 非叶结点中的**多个关键字**均**自小至大**有序排列，即：$K_1 < K_2 < \ldots < K_n$ ；
- A_{i-1} 所指子树上所有关键字均**小于**K_i ；
- A_i 所指子树上所有关键字均**大于**K_i ；

—— 查找树的特性

- 树中所有叶子结点均不带信息，且在树中的同一层次上；
- 根结点或为叶子结点，或至少含有两棵子树；
- 其余所有非叶结点均至少含有$\lceil m/2 \rceil$棵子树，至多含有 m 棵子树；

——平衡树的特性

2.查找过程：

从根结点出发，沿指针**搜索结点**和**在结点内进行**顺序（或折半）**查找** 两个过程交叉进行。

若**查找成功**，则**返回指向**被查关键字所在**结点的指针**和**关键字在结点中的位置**；

若**查找不成功**，则**返回插入位置。**



假设返回的是如下所述结构的记录：

```
typedef struct {
  BTNode  *pt;      // 指向找到的结点的指
  int  i;           // 1..m，在结点中的关键字序号
  int  tag;     // 标志查找成功(=1)或失败(=0
} Result;                   // 在B树的查找结果类型
```

```
Result SearchBTree(BTree T, KeyType K)
  // 在m 阶的B-树 T 中查找关键字 K, 返回
  // 查找结果 (pt, i, tag)。若查找成功，则
  // 特征值 tag=1, 指针 pt 所指结点中第 i
  // 关键字等于 K； 否则特征值 tag=0, 等
  //  K 的关键字应插入在指针 pt 所指结点
  // 中第 i 个关键字和第 i+1个关键字之间
     ...  ...
} // SearchBTree
```



```
p=T;  q=NULL;  found=FALSE;  i=0;
while (p && !found) {
   n=p->keynum;   i=Search(p, K);
   // 在p->key[1..keynum]中查找 i，   p->key[i]<=K<p->key[i+1]
   if (i>0 && p->key[i]==K)  found=TRU
   else { q=p;   p=p->ptr[i]; }  // q 指示 p
}
if (found)  return (p,i,1);          // 查找成功
else return (q,i,0);                 // 查找不成功
```



3．插入

在查找不成功之后，需进行插入。

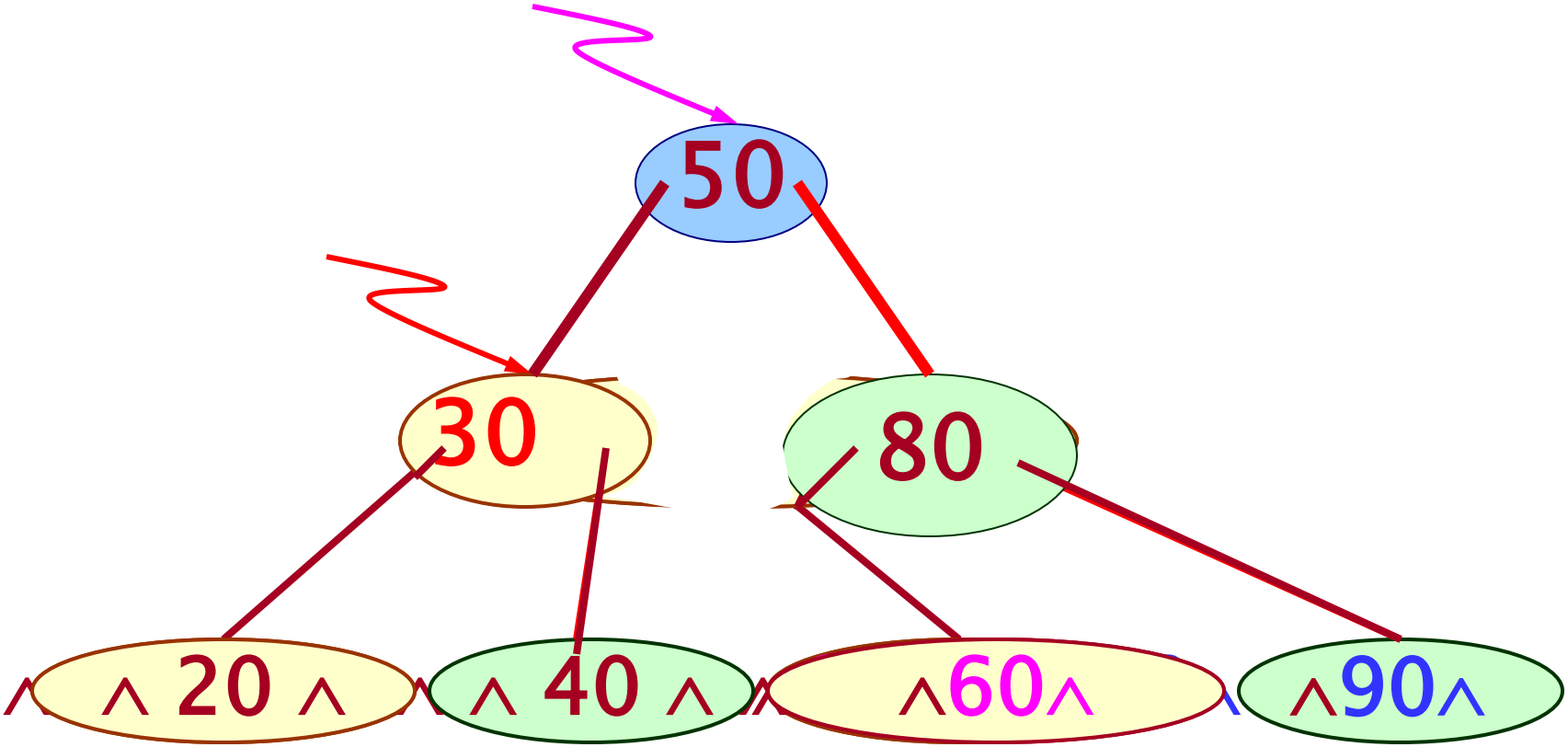
显然，关键字**插入**的**位置**必定在**最下层的非叶结点**，有下列几种情况：

- 1）插入后，该结点的关键字个数n＜m

  不修改指针；例如

- 2）插入后，该结点的关键字个数 n＝m，
  则需进行“结点分裂”，令 s ＝⌈m/2⌉
  在原结点中保留
  $(A_0, K_1, \cdots\cdots, K_{s-1}, A_{s-1})$；
  建新结点
  $(A_s, K_{s+1}, \cdots\cdots, K_n, A_n)$；
  将（K_s，p）插入双亲结点；例如
- 3）若双亲为空，则建新的根结点。
  例如

例如:下列为 3 阶B-树



插入关键字 = 60, 90, 30,

4．删除

和插入的考虑相反，首先必须找到待删关键字所在结点，并且要求删除之后，结点中关键字的个数不能小于⌈m/2⌉–1，否则，要从其左(或右)兄弟结点“借调”关键字，若其左和右兄弟结点均无关键字可借(结点中只有最少量的关键字)，则必须进行结点的“合并”。

5．查找性能的分析

在B-树中进行查找时，其查找时间主要花费在搜索结点（访问外存）上，即主要取决于**B-树的深度**。

问：含 N 个关键字的 *m* 阶 B-树可能达到的最大深度 H 为多少？

反过来问： 深度为H的B-树中，
至少含有多少个结点?

先推导每一层所含最少结点数：

第 1 层	1 个
第 2 层	2 个
第 3 层	$2\times\lceil m/2\rceil$ 个
第 4 层	$2\times(\lceil m/2\rceil)^2$ 个
… …	
第 H+1 层	$2\times(\lceil m/2\rceil)^{H-1}$ 个

假设 m 阶 B-树的深度为 $H+1$，由于第 H+1 层为**叶子**结点，而当前树中含有 N 个关键字，则叶子结点必为 $N+1$ 个，由此可推得下列结果：

$$N+1 \geq 2(\lceil m/2 \rceil)^{H-1}$$

$$H-1 \leq \log_{\lceil m/2 \rceil}((N+1)/2)$$

$$H \leq \log_{\lceil m/2 \rceil}((N+1)/2)+1$$

结论：

在含 N 个关键字的 B–树上进行一次查找，需访问的结点个数不超过

$$\log_{\lceil m/2 \rceil}((N+1)/2)+1$$

四、B^+树

是B-树的一种变型

1．B^+树的结构特点：

※每个叶子结点中含有 *n* 个关键字和 *n* 个指向记录的指针；并且，所有**叶子**结点彼此相**链接**构成一个有序链表，其头指针指向含最小关键字的结点；

※每个非叶结点中的关键字 K_i即为其相应指针 A_i所指子树中关键字的最大值；

※所有叶子结点都处在同一层次上，每个叶子结点中关键字的个数均介于 $\lceil m/2 \rceil$和 m 之间。

2．查找过程

※在 B^+ 树上，既可以进行缩小范围的查找，也可以进行顺序查找；

※在进行缩小范围的查找时，不管成功与否，都必须查到叶子结点才能结束；

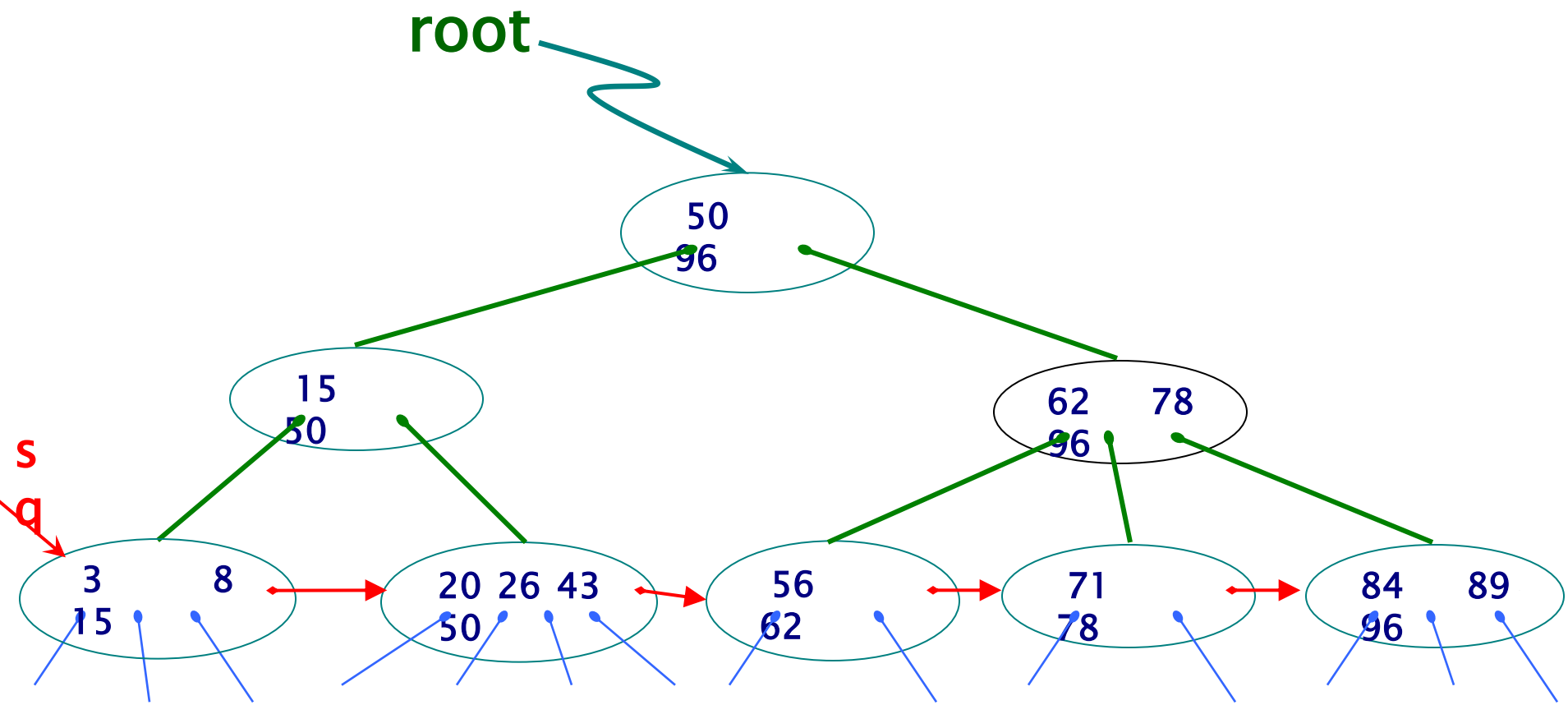
※若在结点内查找时，给定值≤K_i，则应继续在 A_i所指子树中进行查找。



3．插入和删除的操作

类似于B-树进行，即必要时，也需要进行结点的“分裂”或“归并”。




root
50
96
15
50
62
78
96
s
q
3
8
15
20
26
43
50
56
62
71
78
84
89
96

9.3 哈 希 表

一、哈希表是什么?

二、哈希函数的构造方法

三、处理冲突的方法

四、哈希表的查找

五、哈希表的删除操作

六、对静态查找表，．．．



一、哈希表是什么?

? 查找操作要完成什么任务?

待查值k ⟹ 确定k在存储结构中的位置

? 我们学过哪些查找技术? 这些查找技术的共性?

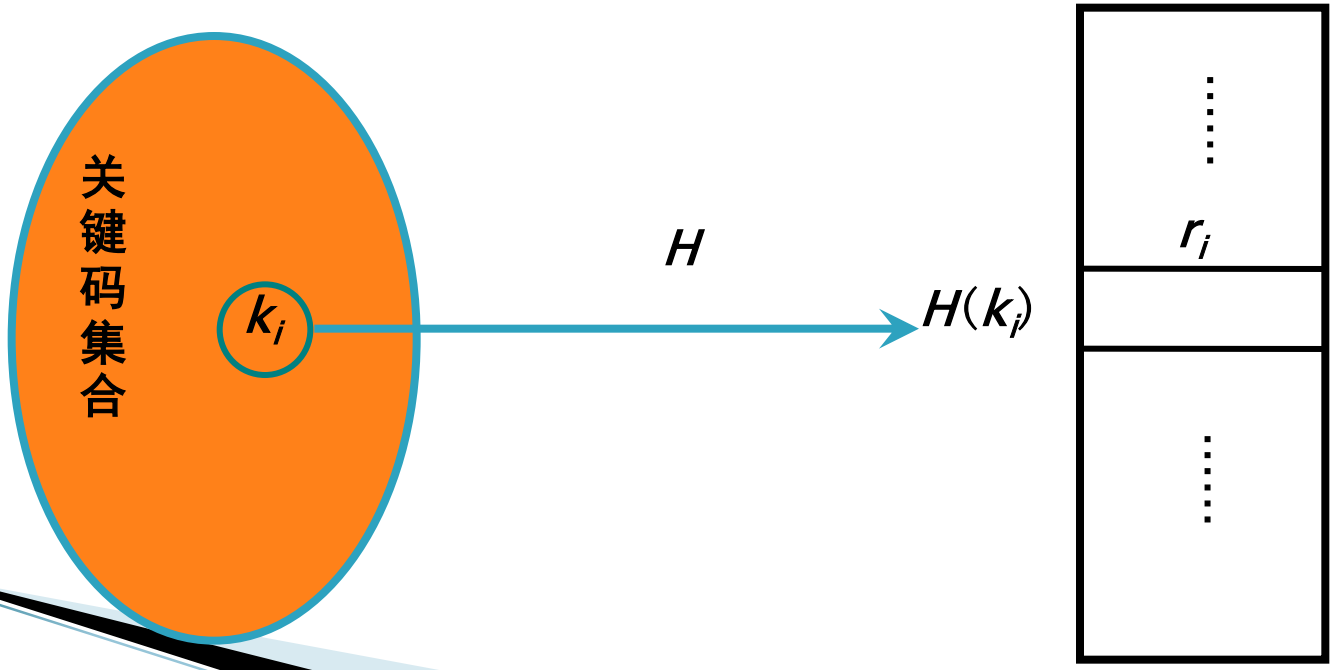
顺序查找、折半查找、二叉排序树查找等。
这些查找技术都是通过一系列的给定值与关键码的比较，查找效率依赖于查找过程中进行的给定值与关键码的比较次数。

? 能否不用比较，通过关键码直接确定存储位置?

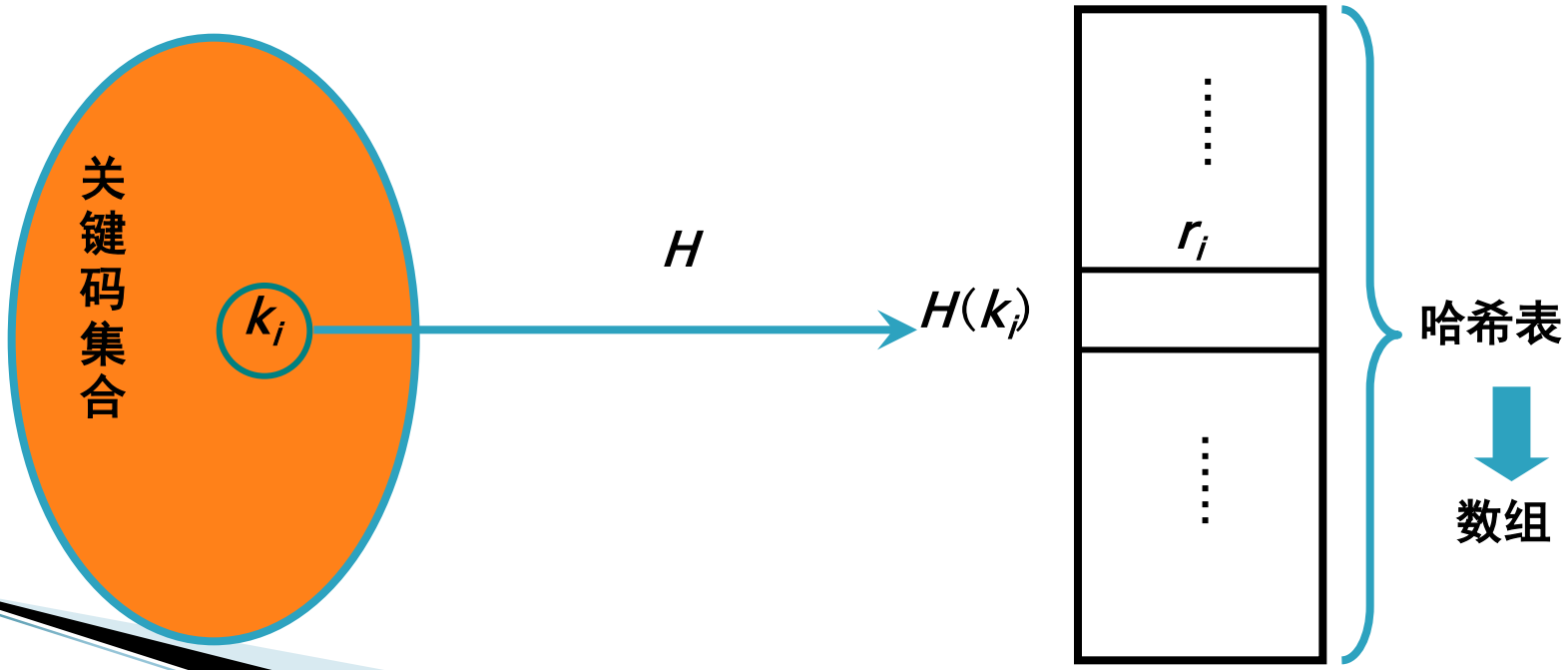
在存储位置和关键码之间建立一个确定的对应关系

概 述

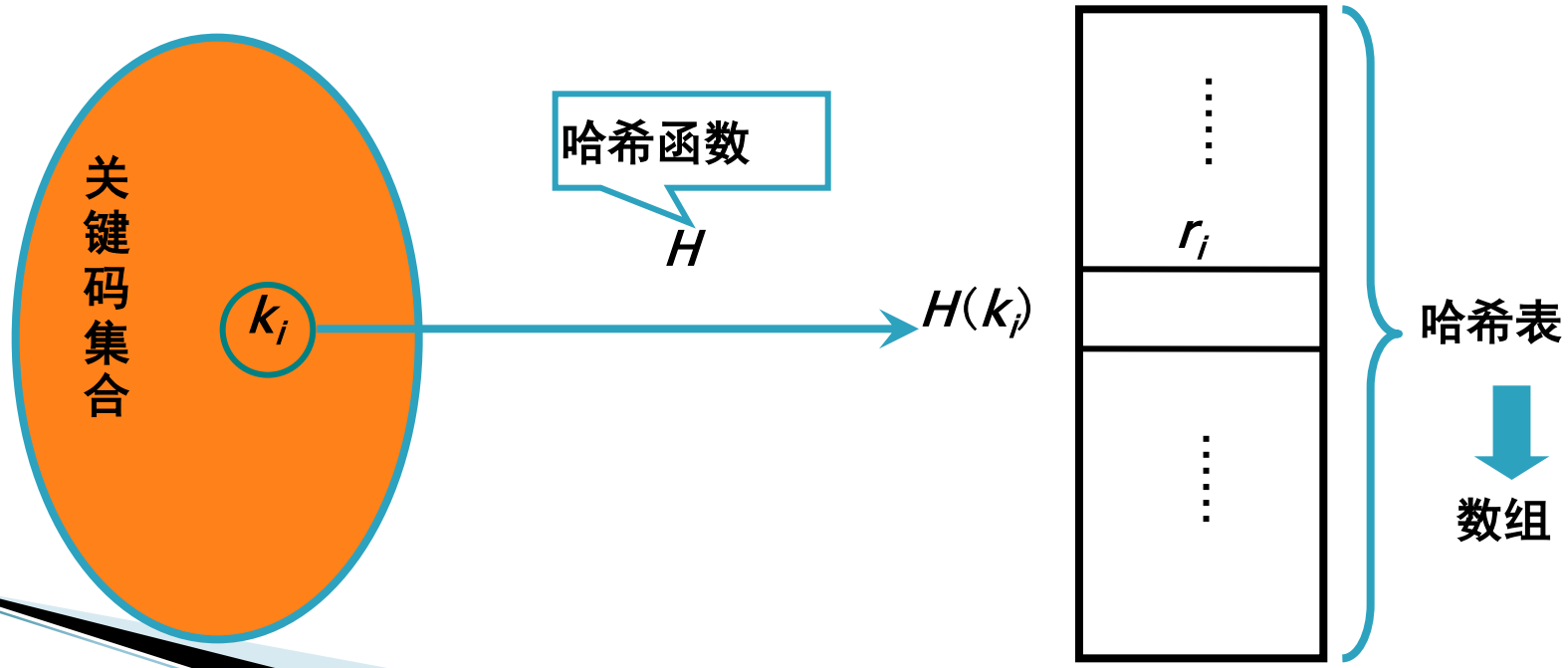
哈希表的基本思想：在记录的存储地址和它的关键码之间建立一个确定的对应关系。这样，不经过比较，一次读取就能得到所查元素的查找方法。



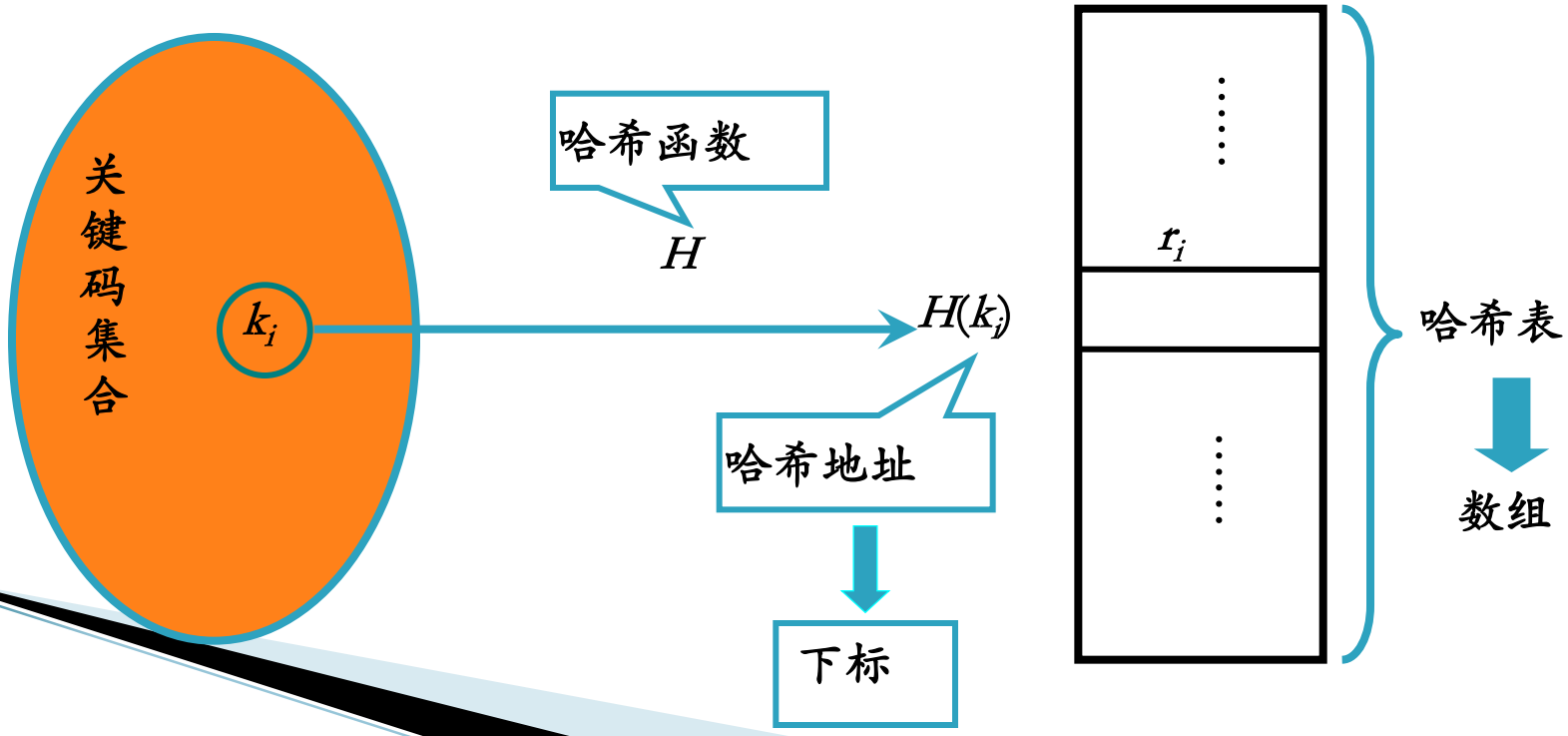
哈希表：采用哈希技术将记录存储在一块连续的存储空间中，这块连续的存储空间称为哈希表。



哈希函数：将关键码映射为哈希表中适当存储位置的函数。



哈希地址： 由哈希函数所得的存储位置址。



? 哈希技术仅仅是一种查找技术吗?

哈希既是一种查找技术，也是一种存储技术。

? 哈希是一种完整的存储结构吗?

哈希只是通过记录的关键码定位该记录，没有完整地表达记录之间的逻辑关系，所以，哈希主要是面向查找的存储结构。

? 哈希技术适合于哪种类型的查找?

哈希技术一般不适用于允许多个记录有同样关键码的情况。哈希方法也不适用于范围查找，换言之，在哈希表中，我们不可能找到最大或最小关键码的记录，也不可能找到在某一范围内的记录。

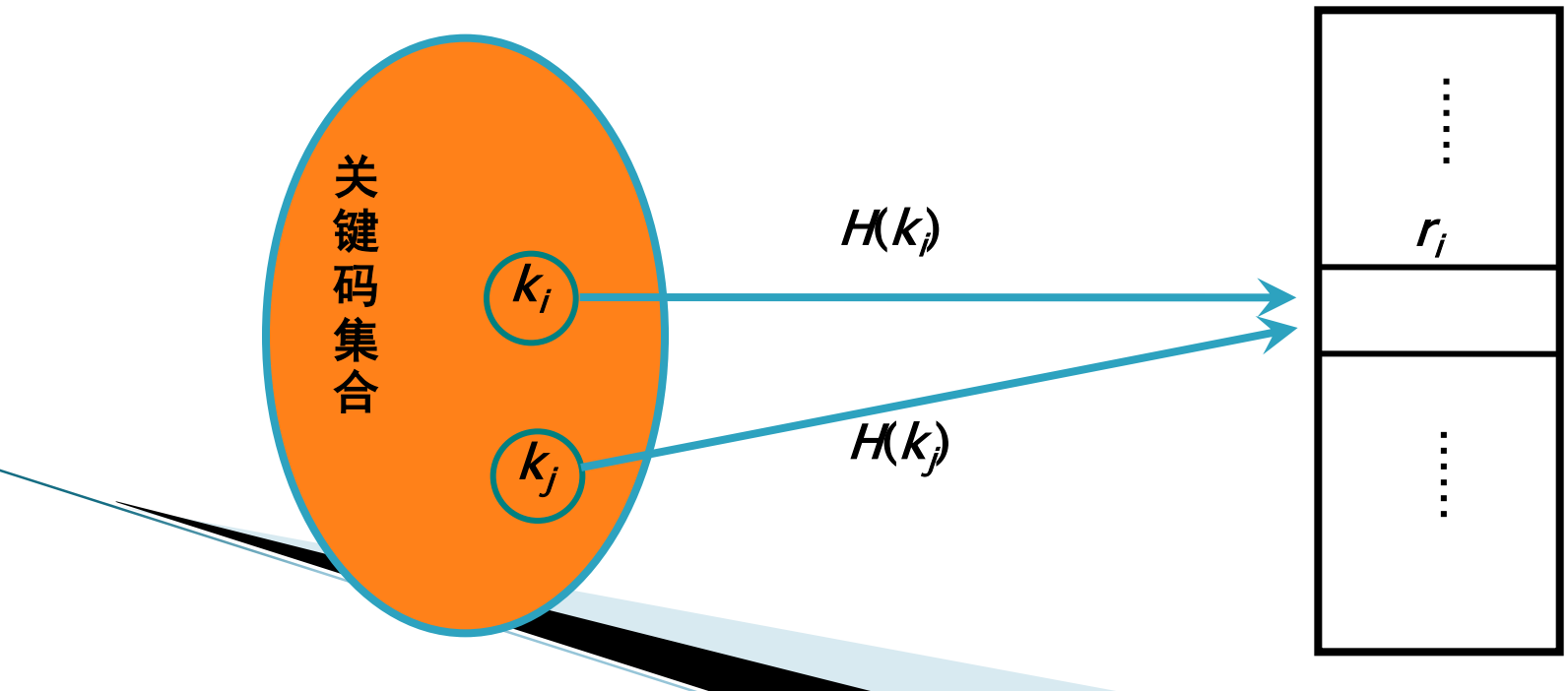
哈希技术最适合回答的问题是：如果有的话，哪个记录的关键码等于待查值。

哈希技术的关键问题：

(1) 哈希函数的设计。如何设计一个简单、均匀、存储利用率高的哈希函数。

(2) 冲突的处理。如何采取合适的处理冲突方法来解决冲突。

冲突：对于两个不同关键码$k_i \neq k_j$，有$H(k_i) = H(k_j)$，即两个不同的记录需要存放在同一个存储位置，k_i和k_j相对于H称做**同义词**。



哈希函数

设计哈希函数一般应遵循以下原则：

(1) 计算简单。哈希函数不应该有很大的计算量，否则会降低查找效率。

(2) 函数值即哈希地址分布均匀。函数值要尽量均匀散布在地址空间，这样才能保证存储空间的有效利用并减少冲突。

例如：对于如下 9 个关键字

{Zhao, Qian, Sun, Li, Wu, Chen, Han, Ye, De

设 哈希函数 f(key) =

⌊(Ord(第一个字母) –Ord('A')+1)/2⌋

0	1	2	3	4	5	6	7	8	9	10	11	12	13
	Chen	Dei		Han		Li		Qian	Sun		Wu	Ye	Zhao

问题：若添加关键字 Zhou，怎么办？

能否找到另一个哈希函数？

从这个例子可见：

1） 哈希函数是一个**映象**，即：

将关键字的集合映射到某个地址集合上，

它的设置很**灵活**，只要这个**地址集**合的大小**不超出允许范围**即可；

2）由于哈希函数是一个**压缩映象**，因此，在一般情况下，很容易产生**“冲突”**现象，即： key1≠ key2，而 f(key1) = f(key2)。

冲突的不可避免性,有它一定的内因。 由于关键字的值域往往比哈希表的个数大得多,所以哈希函数是一种压缩映射, 碰撞是难免的。 例如, 存储100个学生记录, 尽管安排120个地址空间, 但由于学生名(假设不超过10个英文字母)的理论个数超过2610, 要找到一个哈希函数把100个任意的学生名映射成［0, 119］内的不同整数, 实际上是不可能的。

3) **很难**找到一个不产生冲突的哈希函数。一般情况下，只能选择恰当的哈希函数，使冲突尽可能少地产生。

因此，在构造这种特殊的“查找表”时，除了需要选择一个“好”（尽可能少产生冲突）的哈希函数之外；还需要找到一种“处理冲突”的方法。

哈希表的定义：

根据设定的**哈希函数 H(key)** 和所选中的**处理冲突的方法**，将一组关键字**映象到**一个有限的、地址连续的地址集(区间) 上，并以关键字在地址集中的“象”作为相应记录在表中的**存储位置**，如此构造所得的查找表称之为“**哈希表**”。



二、构造哈希函数的方法

对**数字**的关键字可有下列构造方法：

1. 直接定址法　　**4. 折叠法**

2. 数字分析法　　**5. 除留余数法**

3. 平方取中法　　**6. 随机数法**

若是**非数字关键字**，则**需先**对其**进行**

数字化处理

1. 直接定址法

哈希函数为关键字的线性函数

$H(key) = key$　　或者

$H(key) = a \times key + b$

此法仅适合于：

地址集合的大小 = = 关键字集合的大小

2. 数字分析法

假设关键字集合中的每个关键字都是由 s 位数字组成 (u_1, u_2, …, u_s)，分析关键字集中的全体， 并从中提取分布均匀的若干位或它们的组合作为地址。

此方法仅适合于：

能**预先估计出**全体关键字的**每一位上**各种**数字出现的频度。**

3. 平方取中法

以关键字的平方值的中间几位作为存储地址。求“关键字的平方值” 的目的是“扩大差别”，同时平方值的中间各位又能受到整个关键字中各位的影响。

此方法适合于:

关键字中的每一位都有某些数字重复出现频度很高的现象。

3. 平方取中法

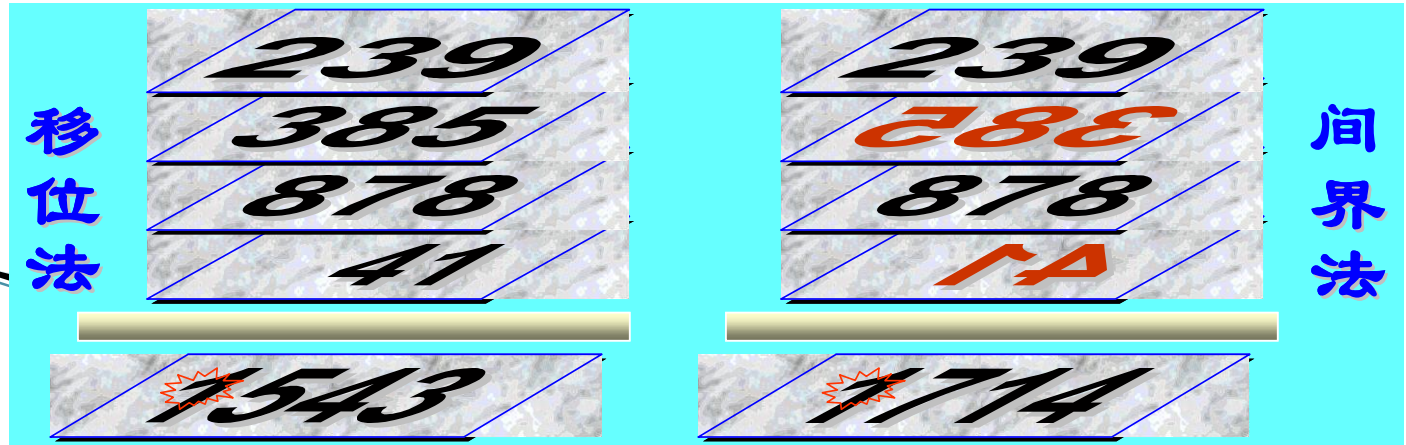
标识符	内码	内码的平方	哈希地址
A	01	01	001
A1	0134	20420	042
A9	0144	23420	342
B	02	4	004
DMAX	04150130	215264436171 00	443

4. 折叠法

将关键字分割成若干部分，然后取它们的叠加和为哈希地址。有两种叠加处理的方法：**移位叠加**和**间界叠加**。

此方法适合于:

关键字的数字位数特别多。



5. 除留余数法

设定哈希函数为:

$$H(key) = key \ MOD \ p$$

其中，　$p \le m$ (表长)　并且

p 应为不大于 m 的素数

或是

不含 20 以下的质因子

为什么要对 p 加限制?

例如:

给定一组关键字为：12, 39, 18, 24, 33, 21，若取 **p=9**, 则他们对应的哈希函数值将为： 3, 3, 0, 6, 6, 3

可见，若 p 中含质因子 3， **则所有含质因子 3 的关键字均映射到“3 的倍数”的地址上**，从而增加了“冲突”的可能。

6.随机数法

设定哈希函数为:

$$H(key) = Random(key)$$

其中，Random 为伪随机函数,要保证函数值是在0～m - 1之间。

通常，此方法用于对长度不等的关键字构造哈希函数。



实际造表时，**采用何种**构造哈希函数的**方法**取决于建表的关键字集合的情况(包括关键字的范围和形态)。**考虑的因素包括：**

1. **计算哈希函数所需时间；**
2. **关键字的长度；**
3. **哈希表的大小；**
4. **关键字的分布情况；**
5. **记录的查找频率**

总的原则是使产生冲突的可能性降到尽可能地小。

三、处理冲突的方法

“**处理冲突**” 的实际含义是：

为产生冲突的地址**寻找下一个**哈希地址。

1. 开放定址法

2. 链地址法

1. 开放定址法

为产生冲突的地址 H(key) 求得一个

地址序列:

$H_0, H_1, H_2, \ldots, H_s$ $\quad 1 \le s \le m-1$

其中: $H_0 = H(key)$

$$H_i = (H(key) + d_i) \text{ MOD } m$$

$$i=1, 2, \ldots, s$$

对增量 d_i 有三种取法：

- 1) 线性探测再散列

  $d_i = c \times i$ 最简单的情况 $c=1$

- 2) 平方探测再散列

  $d_i = 1^2, -1^2, 2^2, -2^2, \ldots,$

- 3) 随机探测再散列

  d_i 是一组**伪随机数列** 或者

  $d_i = i \times H_2(key)$ **（又称双散列函数探测）**

注意：增量 d_i 应具有“完备性”

即：产生的 H_i 均不相同，且所产生的 $s(m-1)$ 个 H_i 值能覆盖哈希表中所有地址。则要求：

※ 平方探测时的表长 m 必为形如 $4j+3$ 的素数（如: 7, 11, 19, 23, … 等）；

※ 随机探测时的 m 和 d_i 没有公因子。



例如：关键字集合

{ 19, 01, 23, 14, 55, 68, 11, 82, 36 }

设定哈希函数 H(key) = key **MOD** 11（表长=11）

若采用**线性探测再散列**处理冲突

0	1	2	3	4	5	6	7	8	9	10
55	01	23	14	68	11	82	36	19		
1	1	2	1	3		6	2	5		1

若采用**二次探测再散列**处理冲突

0	1	2	3	4	5	6	7	8	9	10
55	01	23	14	36	82	68		19		11

$H_2(key)$ 是另设定的一个哈希函数，它的函数值应和 m 互为素数。

若 m 为素数，则 $H_2(key)$ 可以是 1 至 m-1 之间的**任意数**若 m 为 2 的幂次，则 $H_2(key)$ 应是 1 至 m-1 之间的**任意奇数**。

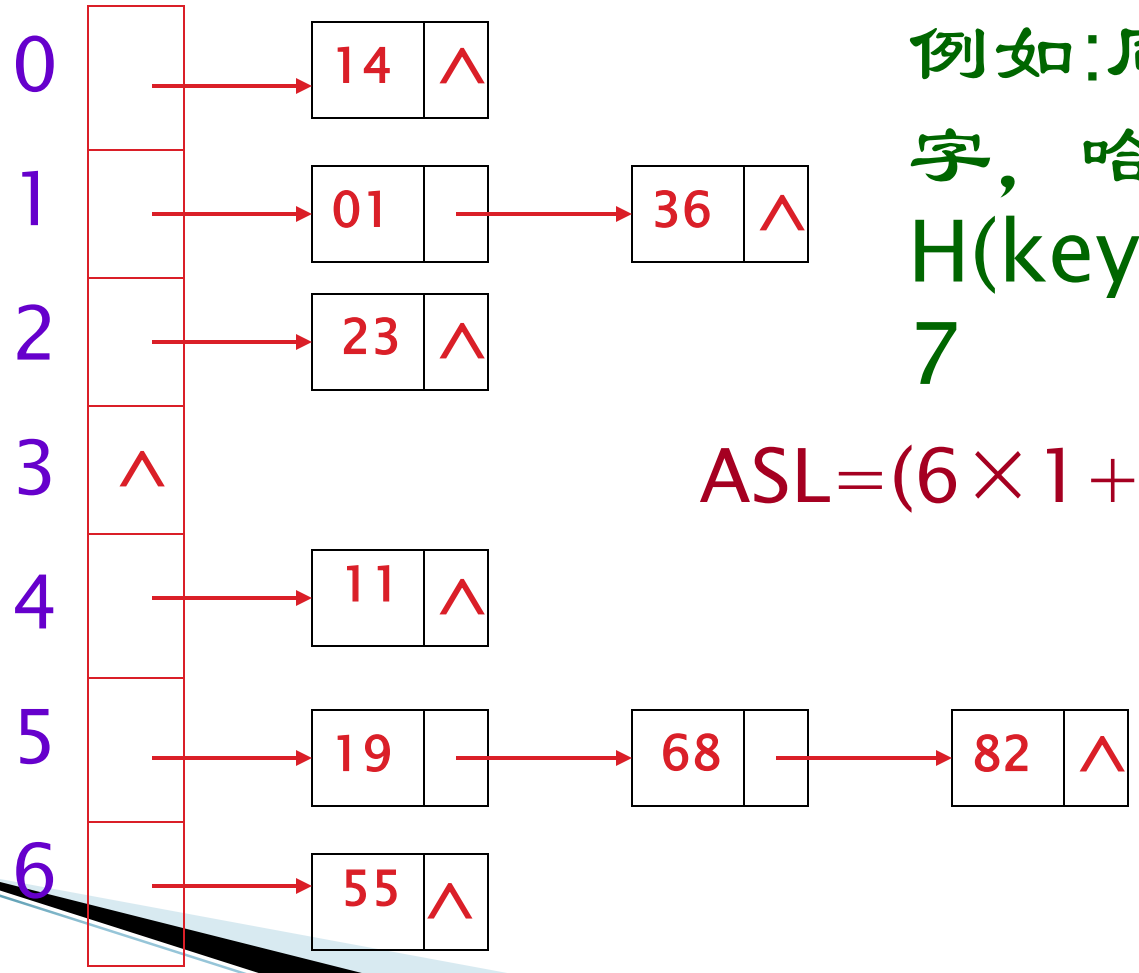
例如，当 m=11时，

可设 $H_2(key)=(3\ key)\ MOD\ 10+1$

0	1	2	3	4	5	6	7	8	9	10
23	01	68	14	11	82	55		19		36
2	1		1	1	2	1		2		1

2. 链地址法

将所有哈希地址相同的记录都链接在同一链表中。



例如:同前例的关键字，哈希函数为H(key)=key MOD 7

ASL=(6×1+2×2+3)/9=13

四、哈希表的查找

查找过程和造表过程一致。假设采用开放定址处理冲突，则**查找过程**为：

对于给定值 K，计算哈希地址 i = H(K)

若 r[i] = NULL 则查找不成功

若 r[i].key = K 则查找成功

否则 “求下一地址 Hi”，直至

r[Hi] = NULL (查找不成功)

或 r[Hi].key = K (查找成功) 为止。

//--- 开放定址哈希表的存储结构 ---

```
int  hashsize[] = { 997, ... };
typedef struct {
  ElemType  *elem;
  int  count;                  // 当前数据元素个数
  int  sizeindex;
                  // hashsize[sizeindex]为当前
} HashTable;
#define  SUCCESS  1
#define  UNSUCCESS  0
#define  DUPLICATE  -1
```

```
Status SearchHash (HashTable H, KeyType K,
                                    int &p, int &c) {
  // 在开放定址哈希表H中查找关键码为K的记录
  p = Hash(K);                  // 求得哈希地址
  while ( H.elem[p].key != NULLKEY &&
                        !EQ(K, H.elem[p].key))
    collision(p, ++c);          // 求得下一探查地址p
  if (EQ(K, H.elem[p].key)) return SUCCESS;
                // 查找成功，返回待查数据元素位置
  else return UNSUCCESS;    // 查找不成功
```

```
Status InsertHash (HashTable &H, Elemtype e
    c = 0;
    if ( HashSearch ( H, e.key, p, c ) == SUCCES
        return DUPLICATE;
            // 表中已有与 e 有相同关键字的元素
    else    if ( c < hashsize[H.sizeindex]/2 ) {
            // 冲突次数 c 未达到上限，（阀值 c 可
    调)          H.elem[p] = e;  ++H.count;
    return OK;
    }else  Recreate(H);  //重建哈希
} //置InsertHash
```

查找不成功时，返回 p 为插入位

哈希表查找的分析：

从查找过程得知，哈希表查找的平均查找长度**实际上并不等于零。**

决定哈希表查找的ASL的因素：

- 1）选用的**哈希函数**；
- 2）选用的**处理冲突的方法**；
- 3）哈希表饱和的程度，**装载因子** $\alpha=n/m$ 值的**大小**（**n—记录数，m—表长度**）

一般情况下，可以认为选用的哈希函数是“均匀”的，则在讨论ASL时，可以不考虑它的因素。

因此，哈希表的ASL是处理冲突方法和装载因子的函数。

例如：前述例子

线性探测处理冲突时，　ASL =22/9

双散列探测处理冲突时，ASL =14/9

链地址法处理冲突时，　ASL =13/9

可以证明：**查找成功**时有下列结果：

线性探测再散列

$$S_{nl} \approx \frac{1}{2}\left(1+\frac{1}{1-\alpha}\right)$$

随机探测再散列

$$S_{nr} \approx -\frac{1}{\alpha}\ln(1-\alpha)$$

链地址法

$$S_{nc} \approx 1+\frac{\alpha}{2}$$

从以上结果可见：

哈希表的平均查找长度是 α 的函数，而不是 n 的函数。

这说明，用哈希表构造查找表时，可以选择一个适当的装填因子 α，使得**平均查找长度限定在某个范围内。**

—— 这是哈希表所特有的特点。



五、哈希表的删除操作

从哈希表中删除记录时，要作**特殊处理**，相应地，需要修改查找的算法。



六、哈希表也可以用来构造静态查找表

并且，**对静态查找表，有时可以**找到不发生冲突的哈希函数。即，此时的**哈希表的 ASL=0**，称此类哈希函数为理想（perfect）的哈希函数。

1. **顺序表**和**有序表**的查找方法及其平均查找长度的计算方法。

2. 静态查找树的构造方法和查找算法,理解静态查找树和折半查找的关系。

3. 熟练掌握**二叉排序树**的构造和查找方法。

4. 理解B-树和B+树的特点以及它们的建树和查找的过程。

5. 熟练掌握哈希表的构造方法，深刻理解哈希表与其它结构的表的实质性的差别。

6. 掌握按定义计算各种查找方法在等概率情况下查找成功时的平均查找长度。